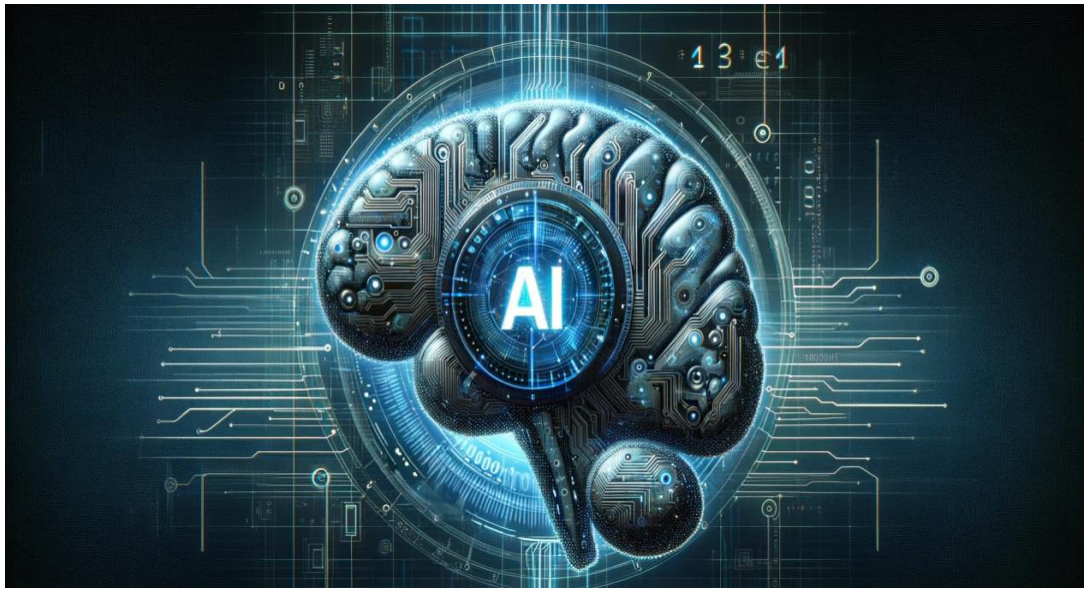
# UNIT 3

# Introduction to AI and Problem Solving/ Searching Techniques

**PREPARED BY**
**Mythili R**
**Dept. of Data Science and Business Systems**




This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# AGENDA

Uniform search strategies

- Breadth First Search
- Depth First Search
- Depth Limited Search
- Bidirectional Search
- Comparing Uniform Search Strategies

Heuristic search strategies

- Greedy Best -First Search
- A* Search
- AO* Search
- Memory Bounded Heuristic Search



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# SEARCH AND CONTROL STRATEGIES

- In Artificial Intelligence, Search techniques are universal problem-solving methods.
- Rational agents or Problem-solving agents in AI mostly used these search strategies or algorithms to solve a specific problem and provide the best result.

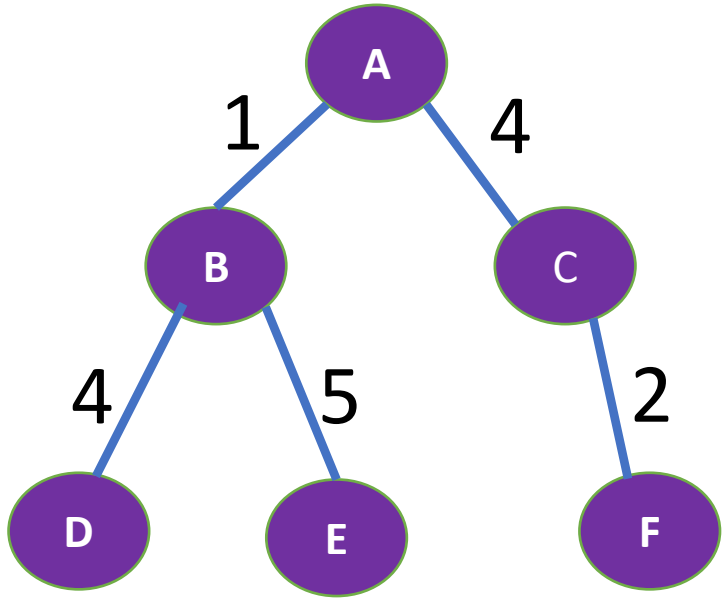
## Search Algorithm Terminologies

- Search: Searching is a step-by-step procedure to solve a search-problem in a given search space.
  - Search Space: Search space represents a set of possible solutions, which a system may have.
  - Start State: It is a state from begins the search.
  - Goal test: It is a function which where agent observe the current state and returns whether the goal state is achieved or not.
- Search tree: A tree representation of search problem is called Search tree. The root of the search tree is the root node which is corresponding to the initial state.
- Actions: It gives the description of all the available actions to the agent.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

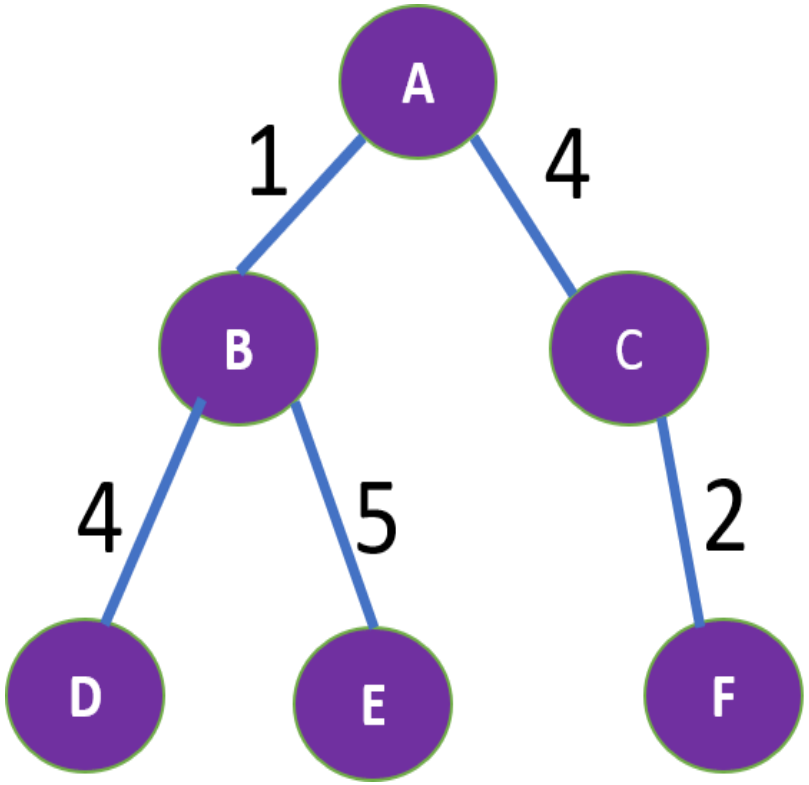
# SEARCH AND CONTROL STRATEGIES

- Transition model: A description of what each action does, can be represented as a transition model.
- Path Cost: It is a function which assigns a numeric cost to each path.
- Solution: It is an action sequence which leads from the start node to the goal node.
- Optimal Solution: If a solution has the lowest cost among all solutions.





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.




**Root Node:** A (initial state)

**Actions from A:**

- Move to B (cost 1)
- Move to C (cost 4)

**Transition Model:**

A → B results in state B
A → C results in state C
B → D results in state D
B → E results in state E
C → E results in state E

**Path Cost:**

Path A → B → D = 1 + 4 = **5**
Path A → B → E = 1 + 5 = 6
Path A → C → F = 4 + 2 = 6

**Solution:** Sequence of actions leading to goal
A → B → D

**Optimal Solution:**

A → B → D (cost = 5) → **lowest among all paths to D**


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Properties of Search Algorithms

- **Completeness**
  - A search algorithm is complete if it guarantees to return a solution whenever one exists for any given input.
- **Optimality**
  - A search algorithm is optimal if the solution it finds is the best (lowest path cost) among all possible solutions.
- **Time Complexity**
  - The amount of time an algorithm takes to complete its task.
- **Space Complexity**
  - The maximum storage space required at any point during the search process.

## Types of Search Algorithms

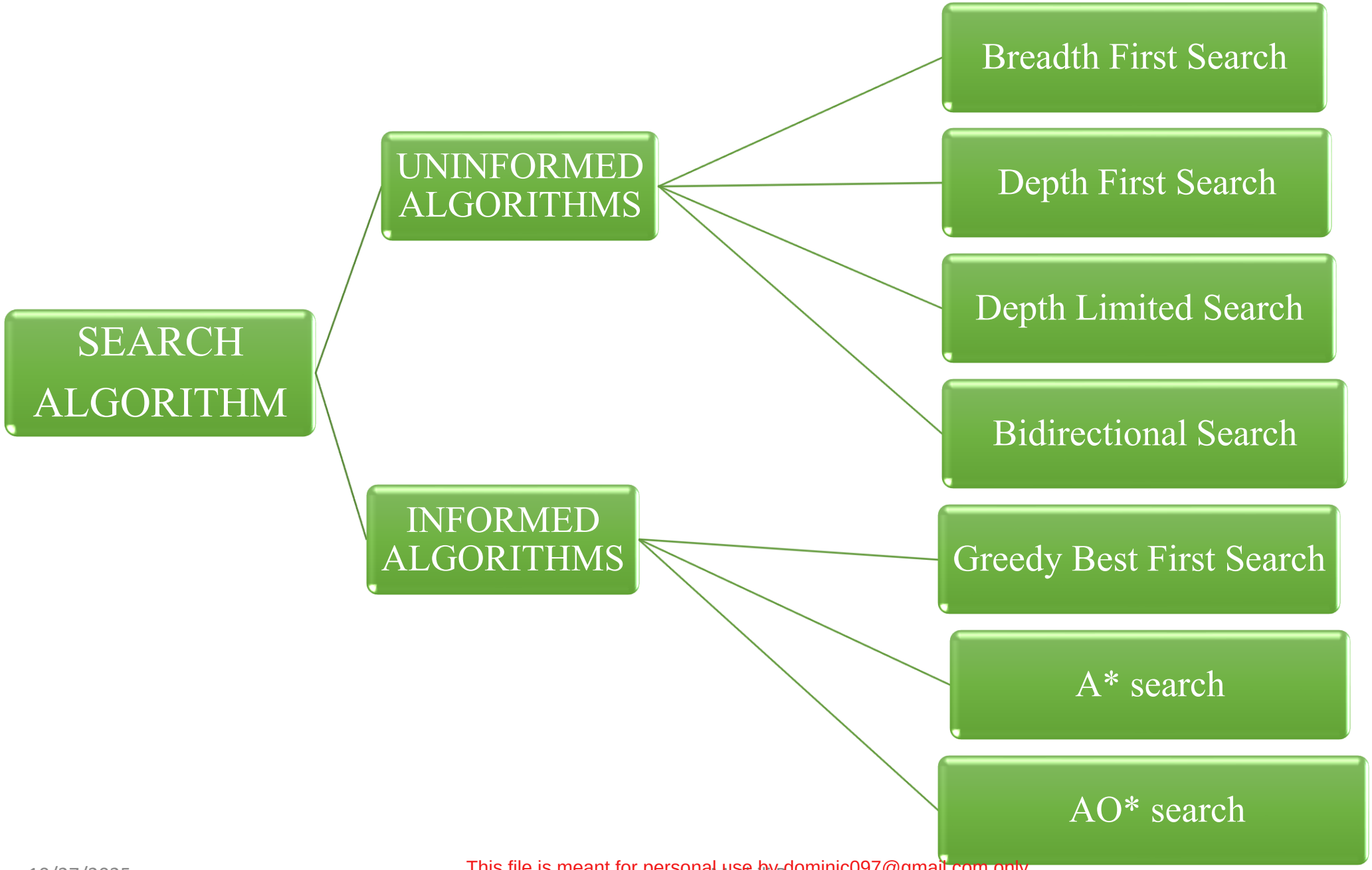
### Uninformed (Blind Search) Algorithms:

- Do not use any domain-specific knowledge, they explore blindly until they find a solution.

### Informed(Heuristic Search) Algorithms:

- Use problem-specific knowledge (heuristics) to find solutions more efficiently.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

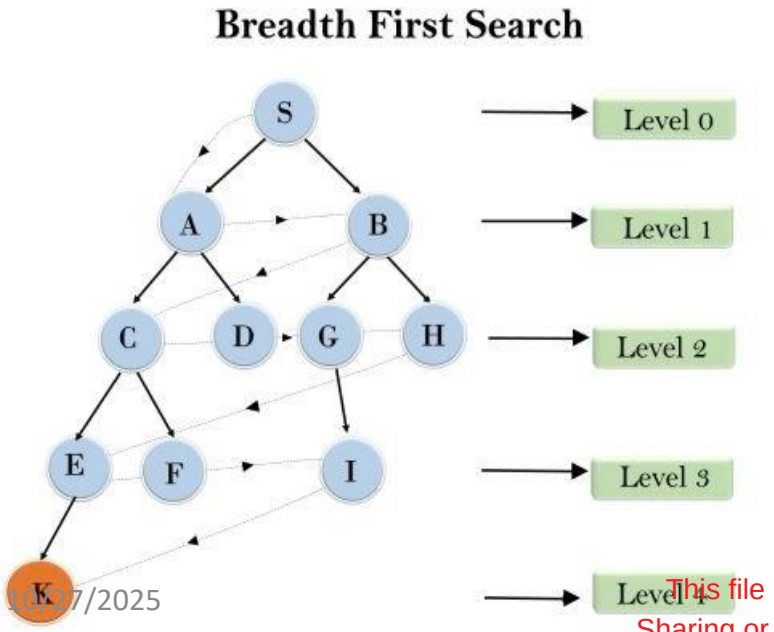





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search (BFS)

- Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadth wise in a tree or graph, so it is called breadth-first search. BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- Breadth-first search implemented using FIFO queue data structure.




In the tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K. BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

**S --> A --> B --> C ---> D --> G -> H --> E --> F --> I --> K**



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

In BFS, the step-wise working is as follows:

| *Queue* | *Check* |
|---|---|
| *A* | — |
| | Removed *A*, is it goal? No, add children. So, *B* and *E* are added. |
| *BE* | |
| *E* | Removed *B*, is it goal? No, add children. |
| | So, *C* is added at the end of queue. |
| *EC* | |
| *C* | Removed *E*, is it goal? No, add children. |
| | So, *F* and *G* are added. |
| *CFG* | |
| *FG* | Removed *C*, is it goal? No, add children. |
| | So, *D* is added. |
| *FGD* | |
| *GD* | Removed *F*, is it goal? No, add children. |
| | Cannot be added, leaf node, so remove the next node. |
| *D* | Removed *G*, is it goal? No, add children. |
| | Cannot be added, leaf node, so remove the next node. |
| Empty | Removed *D*, is it goal? YES! |

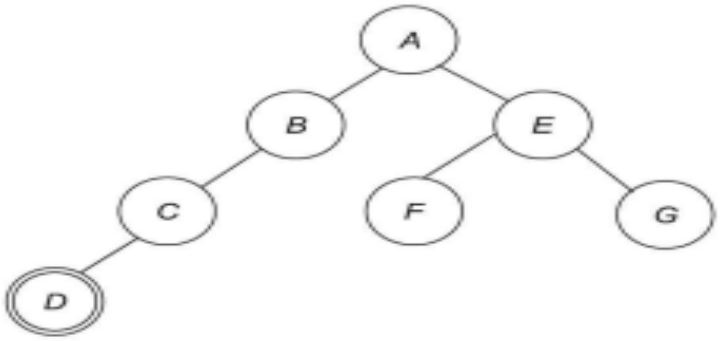
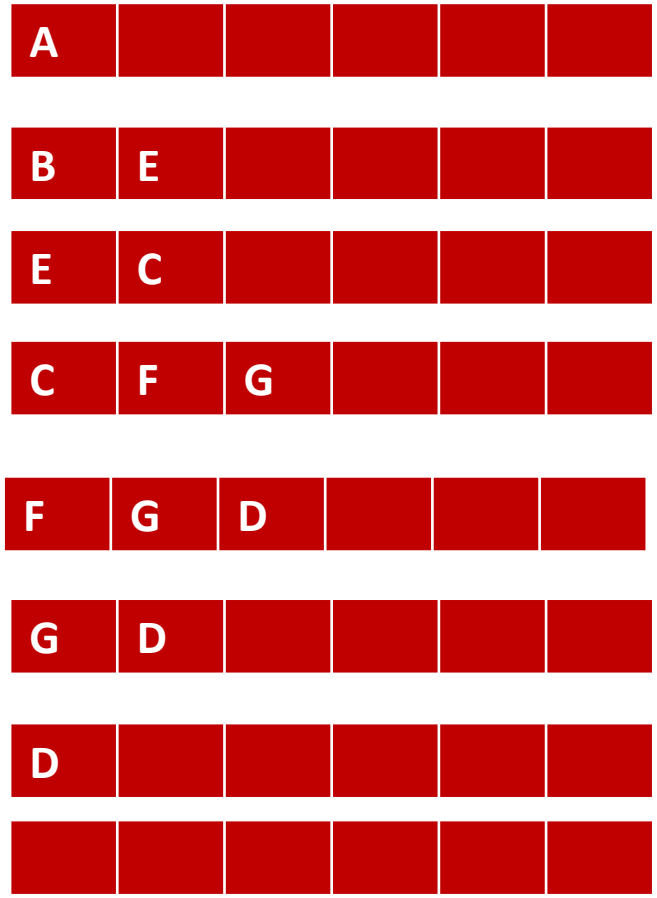



**Figure 3.4** Sample tree input for BFS.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

| Step | Action | Queue | Visited Nodes | Notes |
|---|---|---|---|---|
| Start | Initialize with A | [A] | [A] | Starting point |
| 1 | Dequeue A, enqueue B and E | [B, E] | [A, B, E] | A ≠ D |
| 2 | Dequeue B, enqueue C | [E, C] | [A, B, E, C] | B ≠ D |
| 3 | Dequeue E, enqueue F and G | [C, F, G] | [A, B, E, C, F, G] | E ≠ D |
| 4 | Dequeue C, enqueue D | [F, G, D] | [A, B, E, C, F, G, D] | C ≠ D |
| 5 | Dequeue F (no unvisited children) | [G, D] | [A, B, E, C, F, G, D] | F ≠ D |
| 6 | Dequeue G (no unvisited children) | [D] | [A, B, E, C, F, G, D] | G ≠ D |
| 7 | Dequeue D (Goal found!) | [] | [A, B, E, C, F, G, D] | D = D → **Goal Found!** |



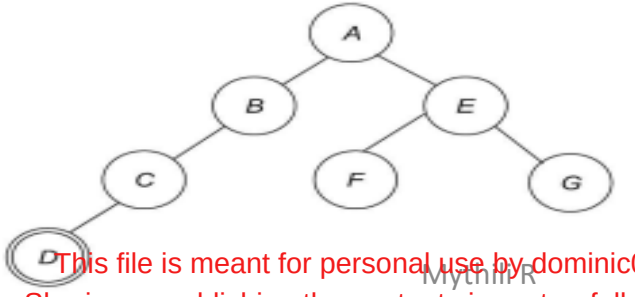




Figure 3.4 Sample tree input for BFS.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

The breadth first search algorithm is summarised as follows:

1. Create a node-list (queue) that initially contains the first node.
2. Do till a goal state is found

   Remove X from node-list. If node-list is empty, then exit.

   Check if this is goal state.

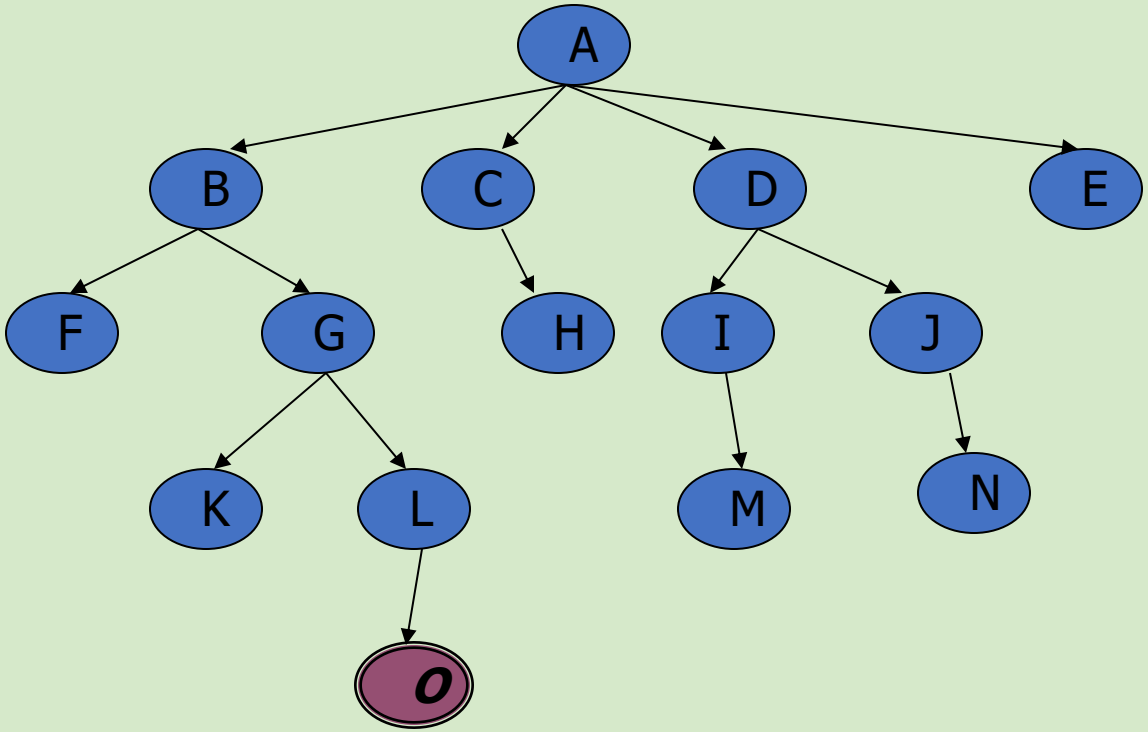
   If yes, return the state.

   If not, get the next level nodes, i.e., nodes reachable from the parent and add to node-list.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Given the following state space (tree search), give the sequence of visited nodes when using BFS (assume that the node ***O*** is the goal state)

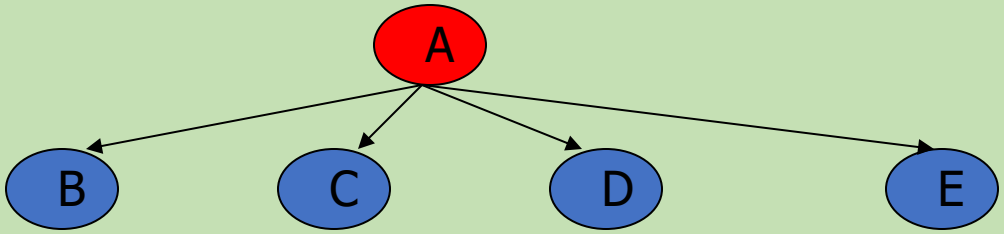





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,

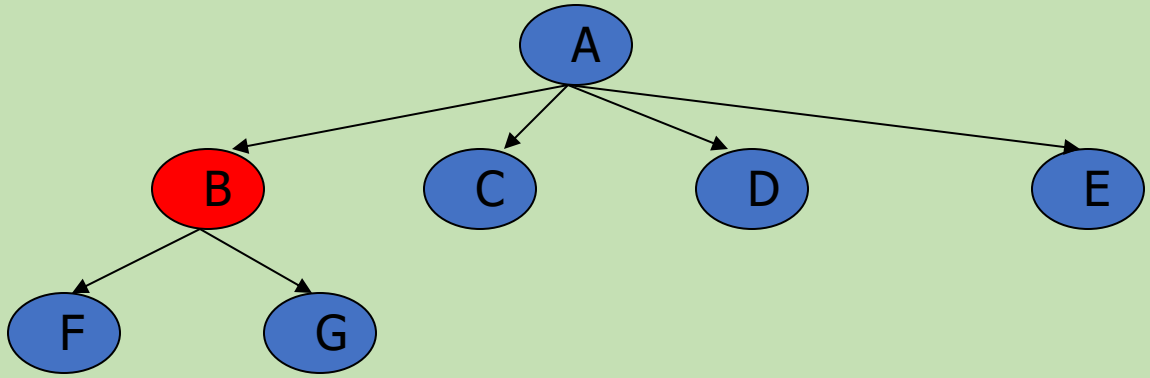




This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,
- B,

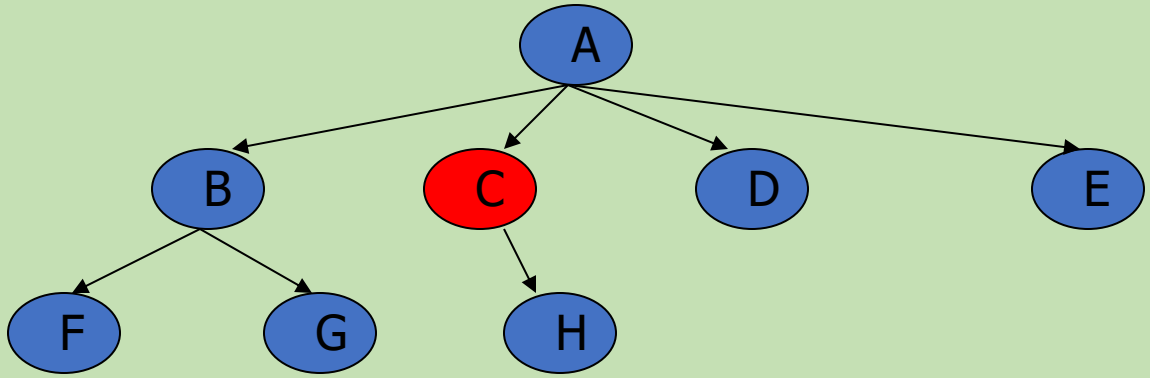




This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,
- B,C

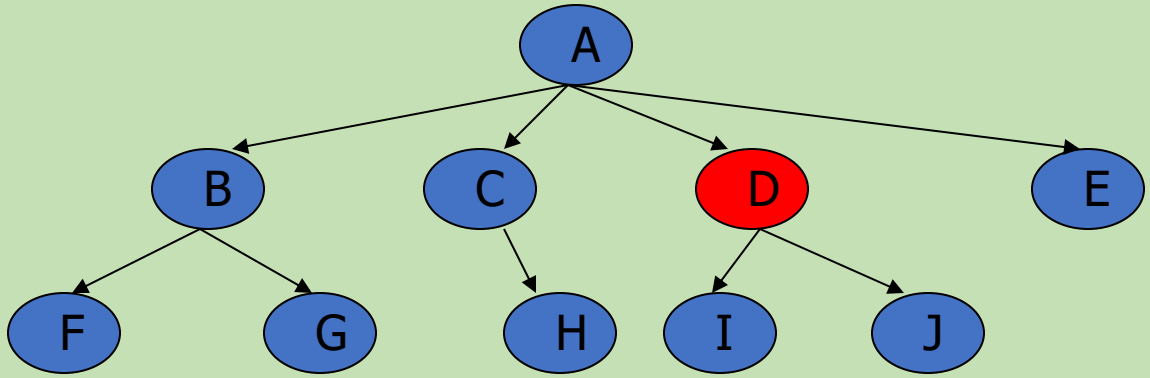




This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,
- B,C,D

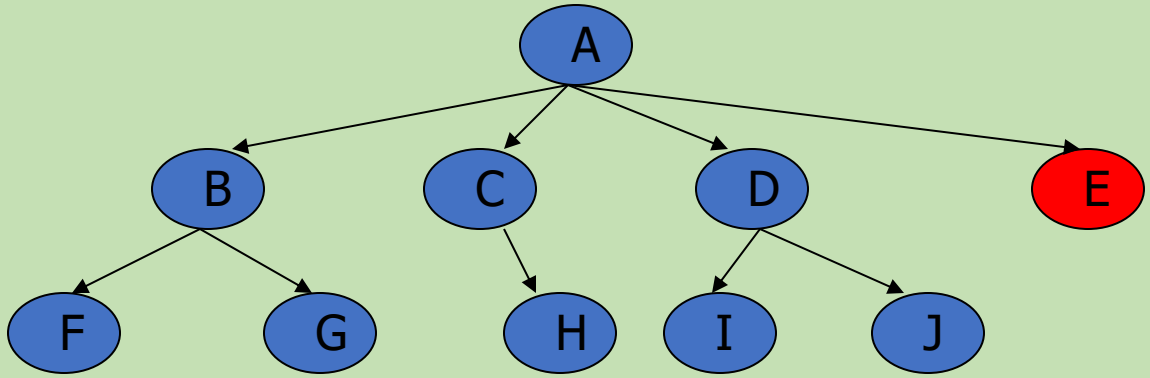




This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,
- B,C,D,E

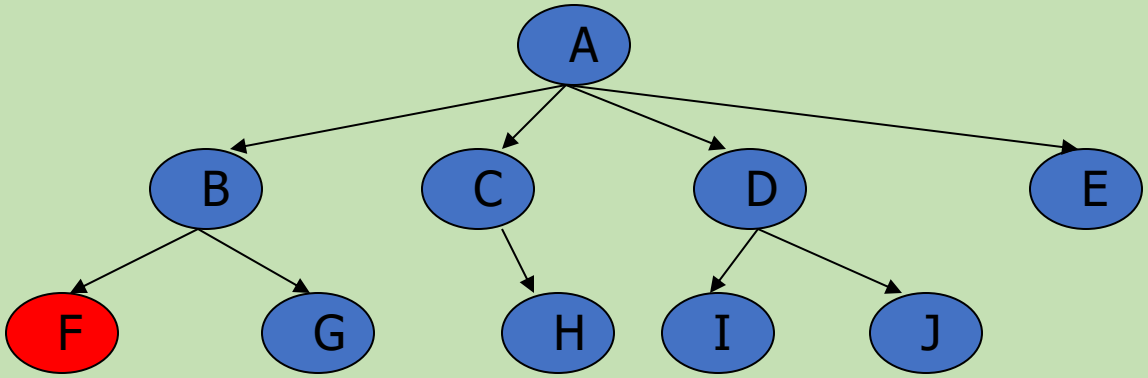




This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,
- B,C,D,E,
- F,

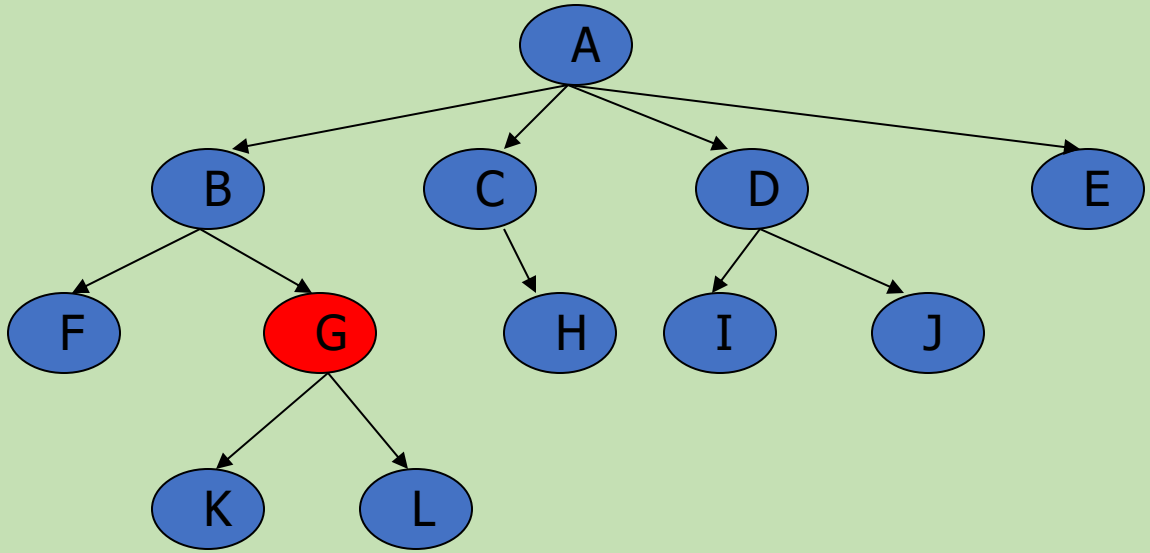





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,
- B,C,D,E,
- F,G

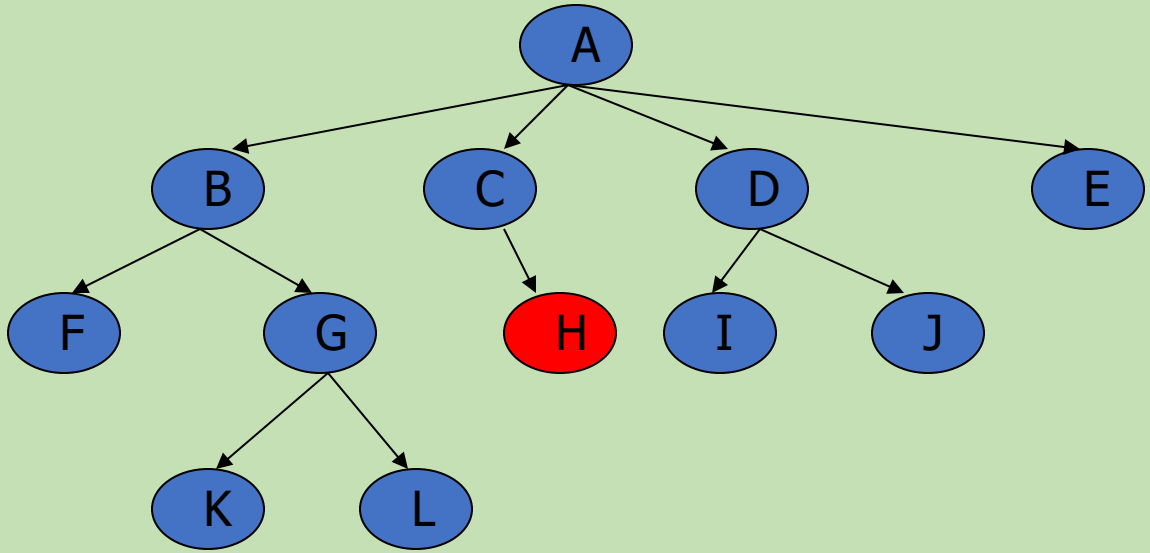




This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,
- B,C,D,E,
- F,G,H

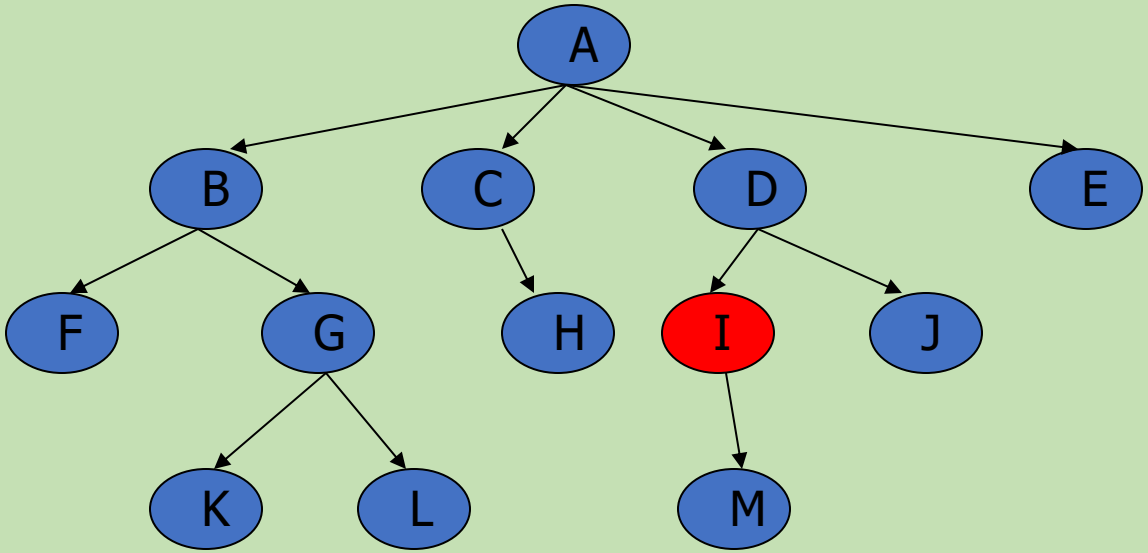




This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,
- B,C,D,E,
- F,G,H,I

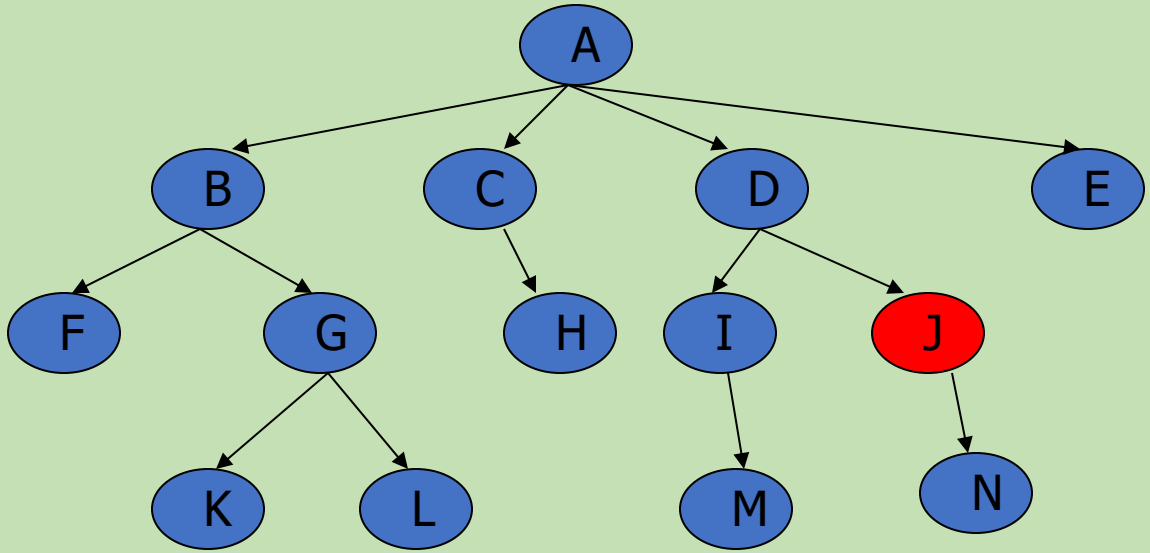




This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,
- B,C,D,E,
- F,G,H,I,J,

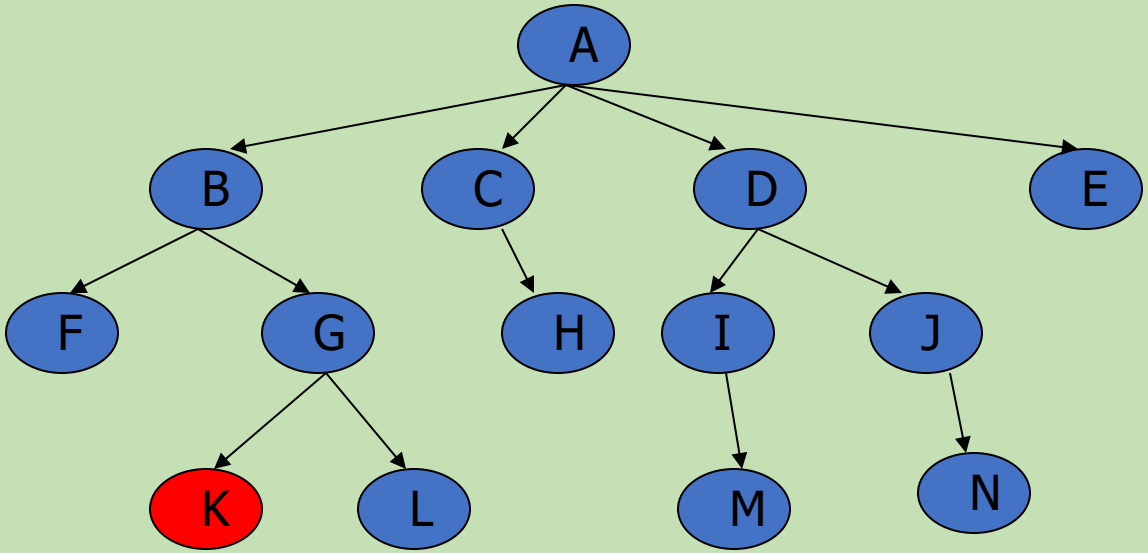




This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,
- B,C,D,E,
- F,G,H,I,J,
- K,

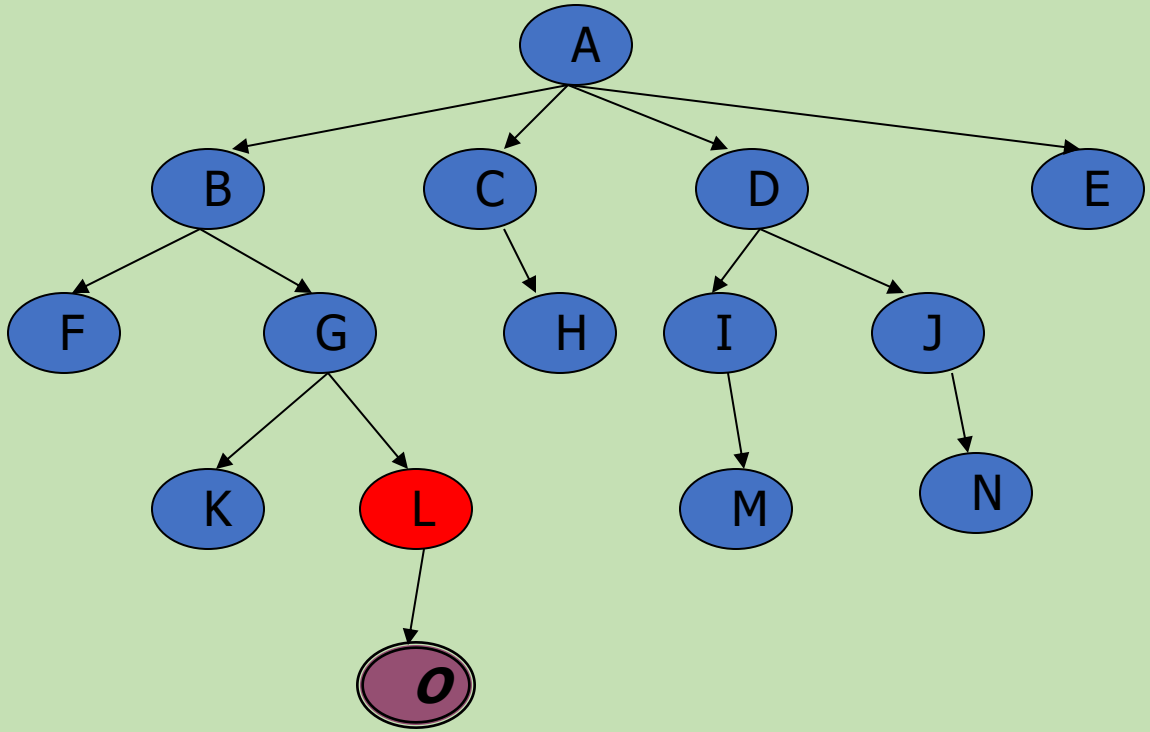





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,
- B,C,D,E,
- F,G,H,I,J,
- K,L

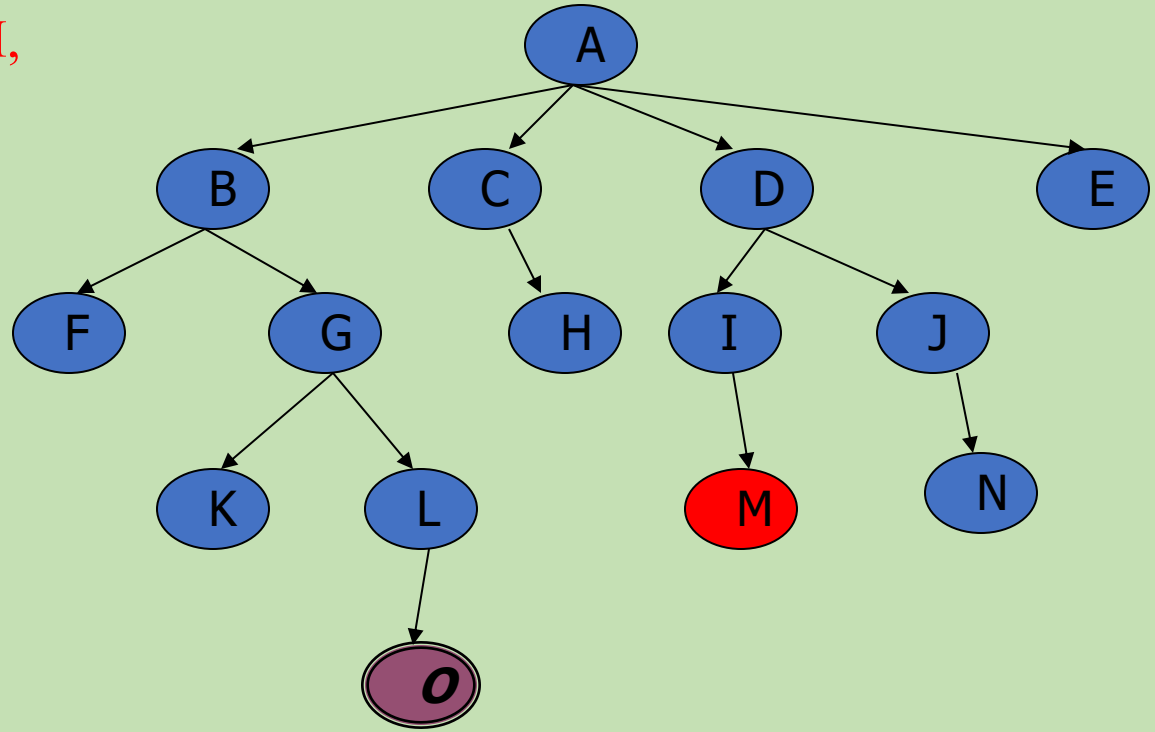





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,
- B,C,D,E,
- F,G,H,I,J,
- K,L, M,

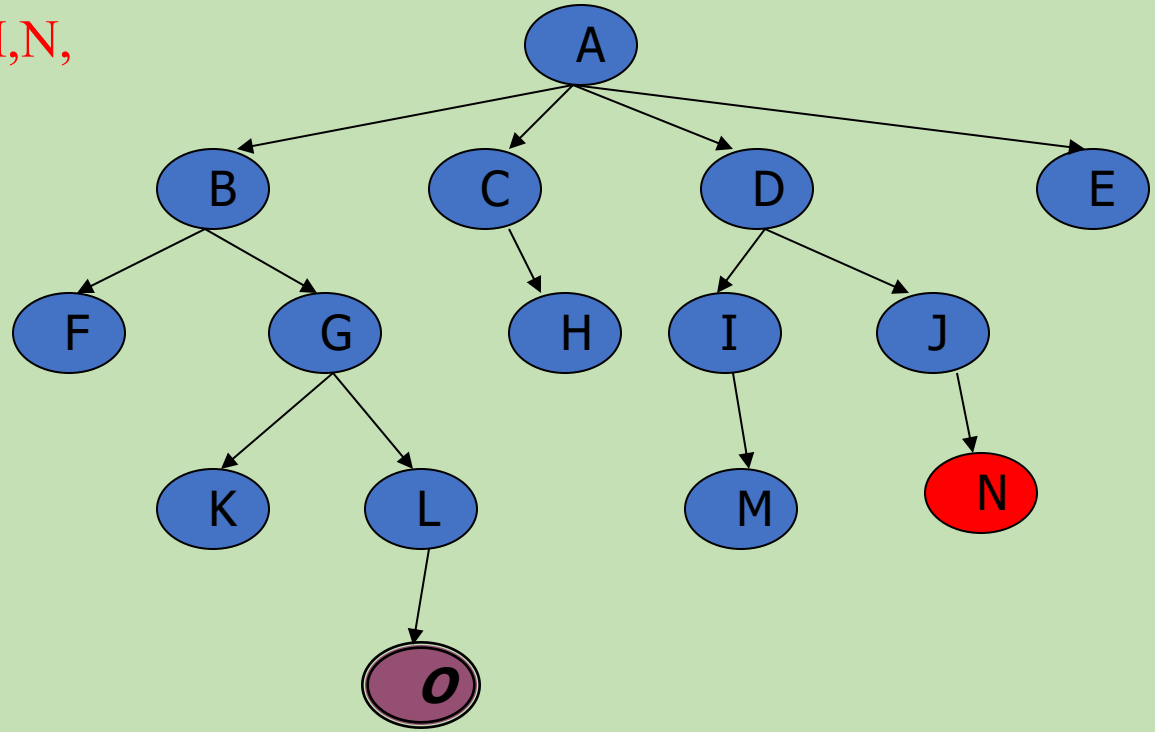




This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,
- B,C,D,E,
- F,G,H,I,J,
- K,L, M,N,

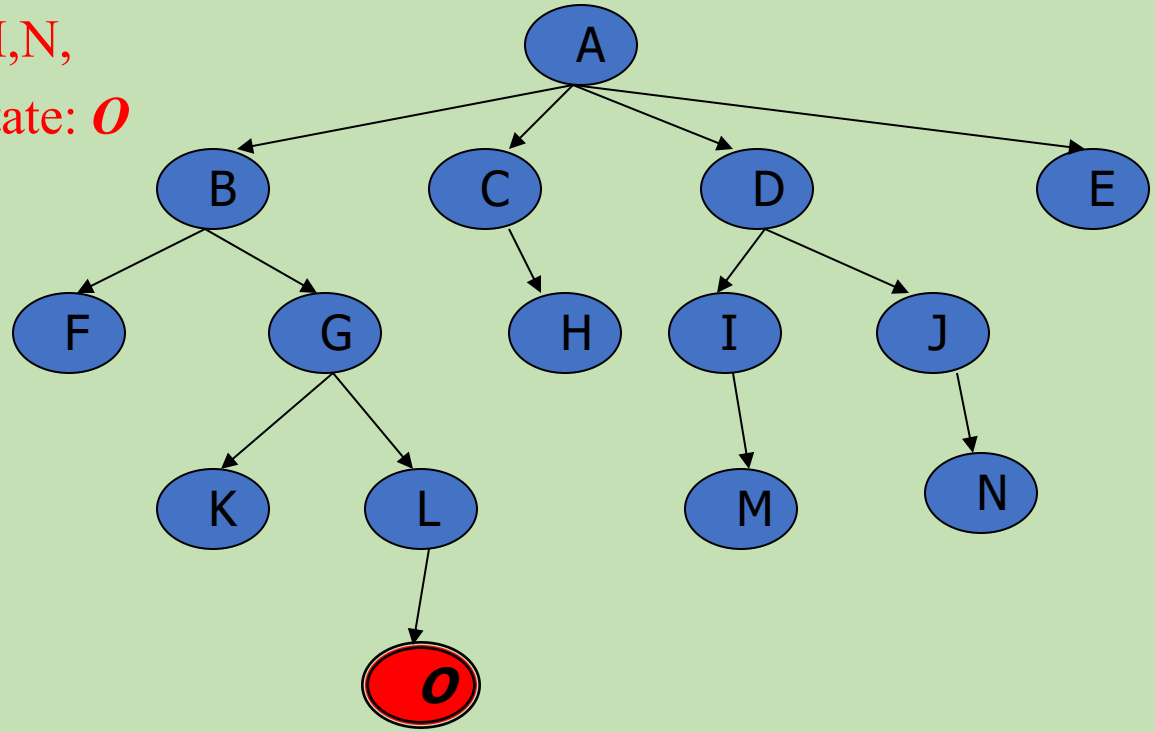




This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- A,
- B,C,D,E,
- F,G,H,I,J,
- K,L, M,N,
- Goal state: ***O***

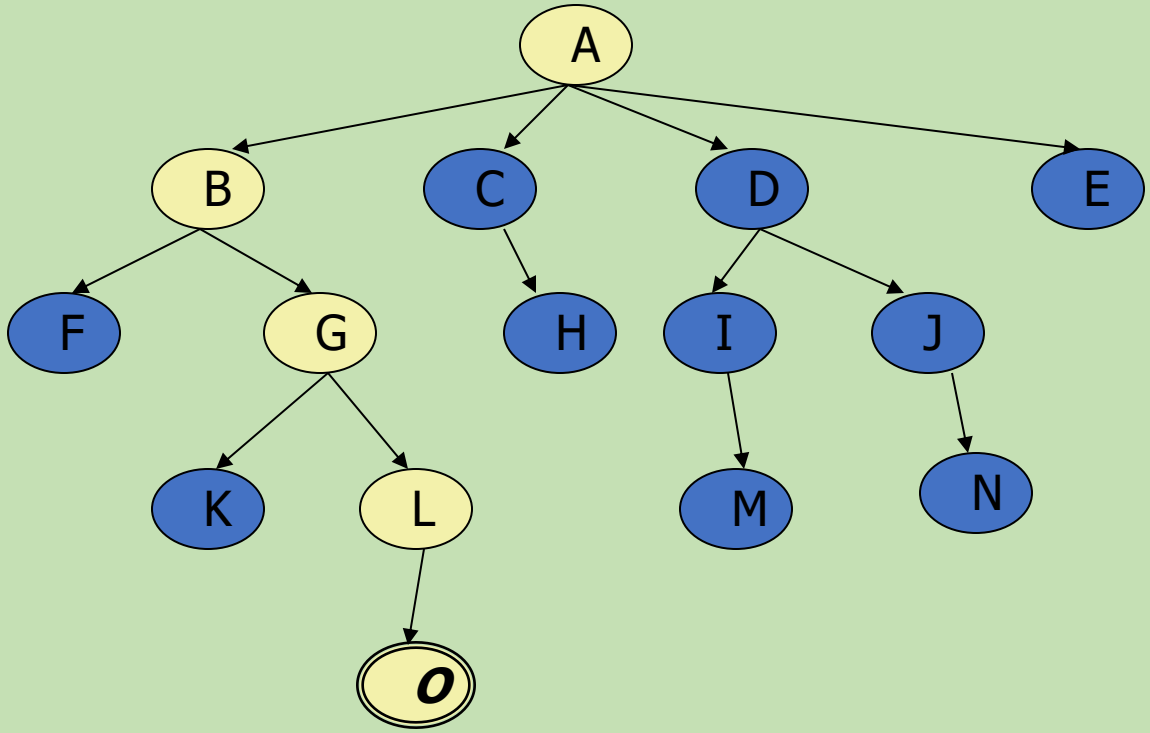





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Breadth First Search

- The returned solution is the sequence of operators in the path: **A, B, G, L, O**






This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# APPLICATIONS OF BFS

- **Social Networking Websites:** In social networks, we can find people within a given distance ‘k’ from a person using Breadth First Search till ‘k’ levels.
- **GPS Navigation systems:** Breadth First Search is used to find all neighboring locations.
- **Broadcasting in Network:** In networks, a broadcasted packet follows Breadth First Search to reach all nodes.

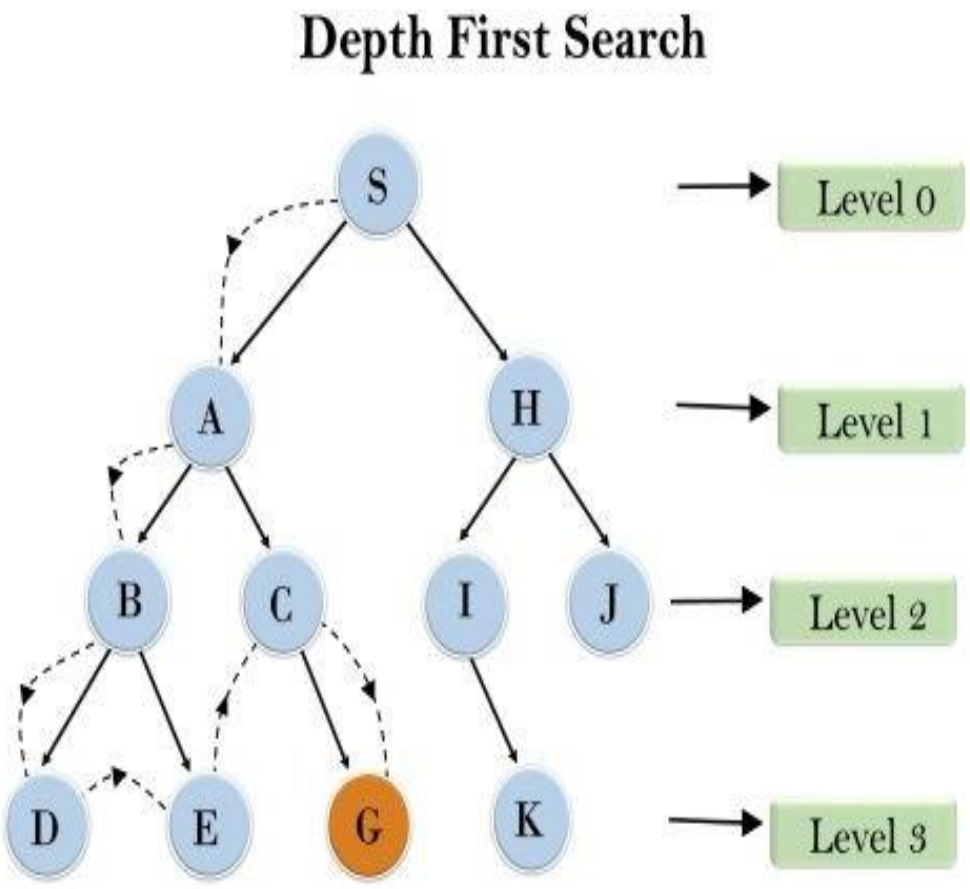

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# DEPTH FIRST SEARCH

- Depth first search (DFS) algorithm starts with the initial node of the graph G, and then goes to deeper and deeper until the goal node or the node which has no children.
- The algorithm, then backtracks from the dead end towards the most recent node that is yet to be completely unexplored.
- The data structure which is being used in DFS is stack. In DFS, the edges that leads to an unvisited node are called discovery edges while the edges that leads to an already visited node are called block edges.
- Depth-first search (DFS) is an algorithm for traversing or searching tree or graph data structures.
- The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth First Search (DFS)




- **DFS** explores the graph by going as **deeply as possible** along each branch before backtracking.
- It uses a **Stack (LIFO)** to manage the order of nodes to visit.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

With the start node of $A$ and the goal state of $D$, we have the DFS approach carried out as follows:

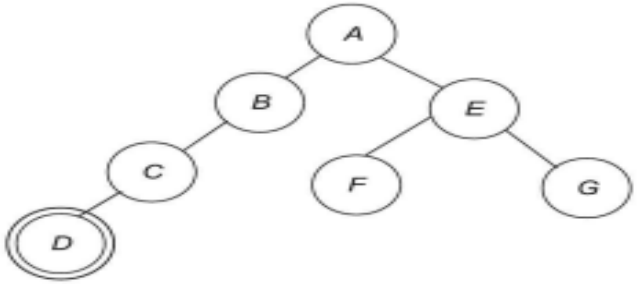



**Figure 3.7** Sample tree input for DFS.

| | Stack | |
|---|---|---|
| **Step 1:** | $A \leftarrow$ top<br>$B \leftarrow$ top $E$ | Pop $A$, is it goal? No, push its children on the stack.<br>$A$'s successor is pushed. |
| **Step 2:** | $B \leftarrow$ top $E$<br>$C \leftarrow$ top $E$ | Pop $B$, is it goal? No, push its children on the stack.<br>$B$'s successor is pushed. |
| **Step 3:** | $C \leftarrow$ top<br>$D \leftarrow$ top $E$ | Pop $C$, is it goal? No, push its children on the stack.<br>$C$'s successor is pushed. |
| **Step 4:** | $D \leftarrow$ top $E$ | Pop $D$, is it goal?? YES! |

Pop all the contents. Path found!

So, by traversing from one path in one direction of depth, we lead to the goal state. In this case, no backtracking is required. Consider the second case, where we want to reach to $F$.

The initial working is same as the way we had to reach to $D$. The example below shows that after having reached till $D$ state, the next step needs backtracking and hence it starts with $E$. So, we will start from that point. Thus, the stack contents are as follows:

| | Stack | |
|---|---|---|
| **Step 1:** | $E \leftarrow$ top | After having traversed the steps till $D$, stack will have $E$.<br>So, pop $E$, is it goal? No, push its children. |
| **Step 2:** | $F \leftarrow$ top $G$ | E's successor is added. |
| **Step 3:** | $F \leftarrow$ top $G$<br>$G \leftarrow$ top | Pop $F$, is it goal?? YES! |

Pop the contents. Path found! So, the path $A$-$E$-$F$ is found.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

| Step | Action | Stack (Top → Bottom) | Visited Nodes | Notes |
|---|---|---|---|---|
| Start | Push A | [A] | [] | Start from A |
| 1 | Pop A → Push E, B | [E, B] | [A] | Visit A |
| 2 | Pop B → Push C | [E, C] | [A, B] | Visit B |
| 3 | Pop C → Push D | [E, D] | [A, B, C] | Visit C |
| 4 | Pop D | [E] | [A, B, C, D] | **Goal Found!** |

**DFS Final Path: A → B → C → D**


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

| Step | Action | Stack (Top → Bottom) | Visited Nodes | Notes |
|---|---|---|---|---|
| Start | Push A | [A] | [] | Start from A |
| 1 | Pop A → Push E, B | [E, B] | [A] | Visit A |
| 2 | Pop B → Push C | [E, C] | [A, B] | Visit B |
| 3 | Pop C → Push D | [E, D] | [A, B, C] | Visit C |
| 4 | Pop D | [E] | [A, B, C, D] | D has no children |
| 5 | Pop E → Push G, F | [G, F] | [A, B, C, D, E] | Visit E |
| 6 | Pop F | [G] | [A, B, C, D, E, F] | **Goal Found!** |

**A → E → F**

Due to DFS order, the algorithm *tried* the entire **A → B → C → D** path first before backtracking and going to **E → F**.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

**TABLE 3.1** Advantages of Depth First Search and Breadth First Search

| *Advantages of Depth First Search* | *Advantages of Breadth First Search* |
|---|---|
| 1. Simple to implement<br>2. Needs relatively small memory for storing the state-space | 1. Guaranteed to find a solution (if one exists)<br>2. Depending on the problem, can be guaranteed to find an optimal solution |

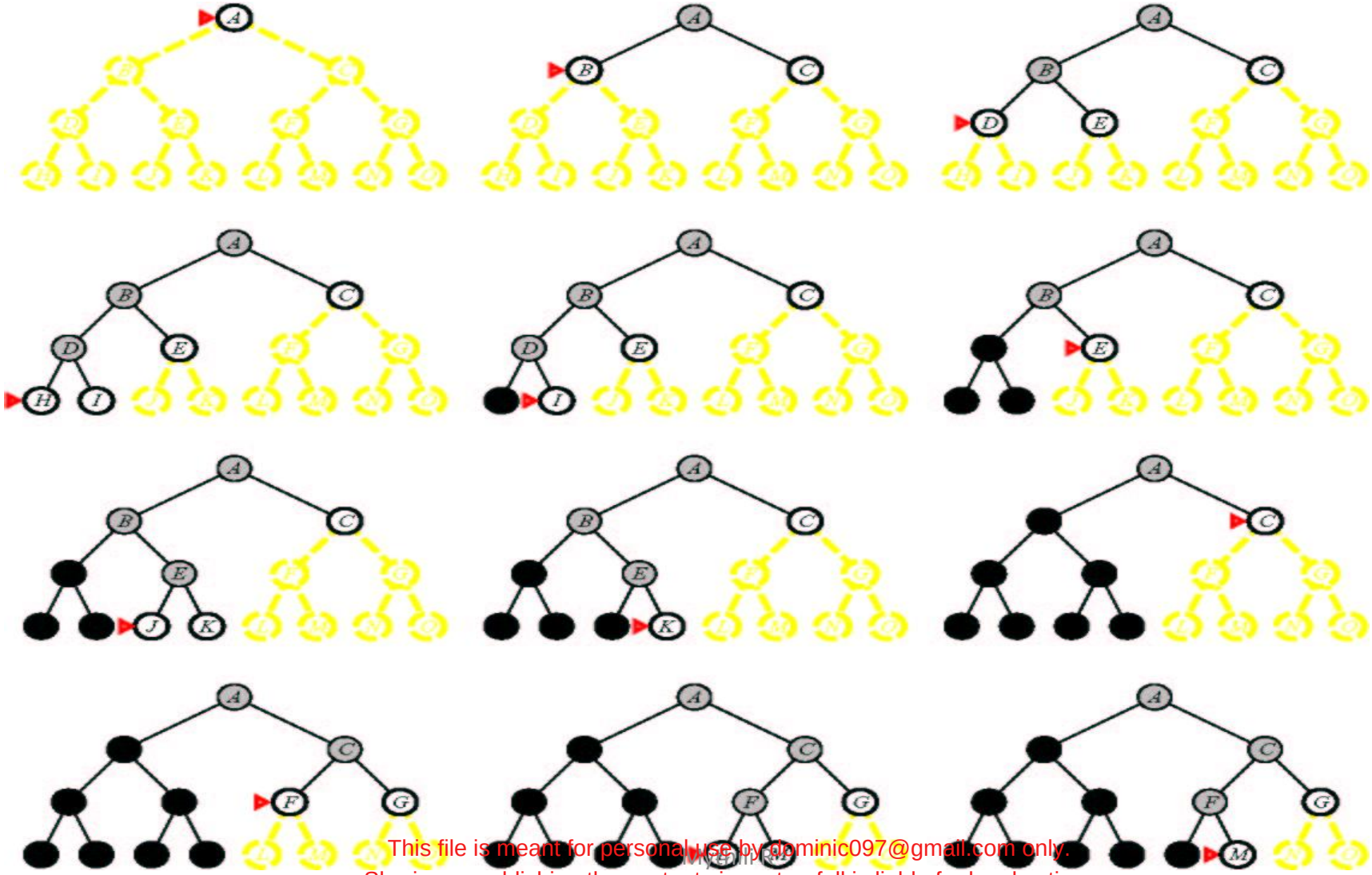
**TABLE 3.2** Disadvantages of Depth First Search and Breadth First Search

| *Disadvantages of Depth First Search* | *Disadvantages of Breadth First Search* |
|---|---|
| 1. Cannot find solution in all cases, and hence, can sometimes fail to find a solution<br>2. Not guaranteed to find an optimal solution can take a lot of time to find a solution | 1. Memory management along with allocation is the key factor, and hence, more complex to implement<br>2. Needs a lot of memory for storing the state space if the search space has a high branching factor |



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth First Search





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth-First Search Algorithm

1. If the initial state is a goal state, quit and return success.
2. Otherwise, do the following until success or failure is signaled:
   a) Generate a successor, E, of the initial state. If there are no more successors, signal failure.
   b) Call Depth-First Search with E as the initial state.
   c) If success is returned, signal success. Otherwise continue in this loop.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- **Completeness:** DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.
- **Time Complexity:** Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$$T(b)= 1+ b^2+ b^3 +.....+ b^d=O(b^d)$$

**Where, b=branching factor**

**d= maximum depth of any node.**

- **Space Complexity:** DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is **O(bd)**.
- **Optimal:** DFS search algorithm is non-optimal, as it may generate a large number of steps or high cost to reach to the goal node.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

## Advantages of Depth-First Search (DFS):

### Low Memory Usage:

DFS requires **less memory** because it only needs to store the nodes **on the current path** (not all siblings or levels, like in BFS).

### Potentially Faster Path Discovery:

DFS may **find a solution quickly** without exploring the entire search space especially if the solution is **deep** in the tree.

## Disadvantages of Depth-First Search (DFS):

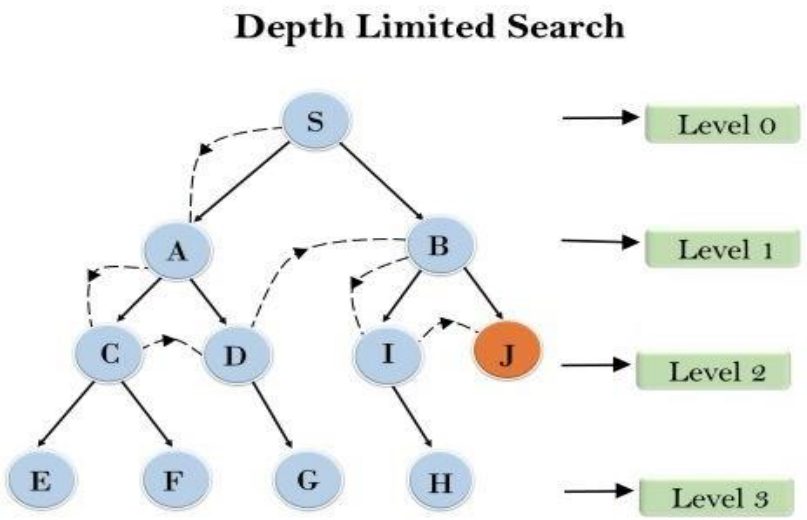
### May Find a Sub-optimal Solution:

DFS might return a solution that is **not the best** (e.g., longer or more costly), since it doesn't guarantee the **shortest path** like BFS.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth Limited Search

- A depth-limited search algorithm is similar to depth-first search with a predetermined limit.
- Depth-limited search can solve the drawback of the infinite path in the Depth-first search.
- In this algorithm, the node at the depth limit will treat as it has no successor nodes further.





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

It is simply DFS with a depth bound.

- Searching is not permitted beyond the depth bound.
- Works well if we know what the depth of the solution is.
- Termination is guaranteed.

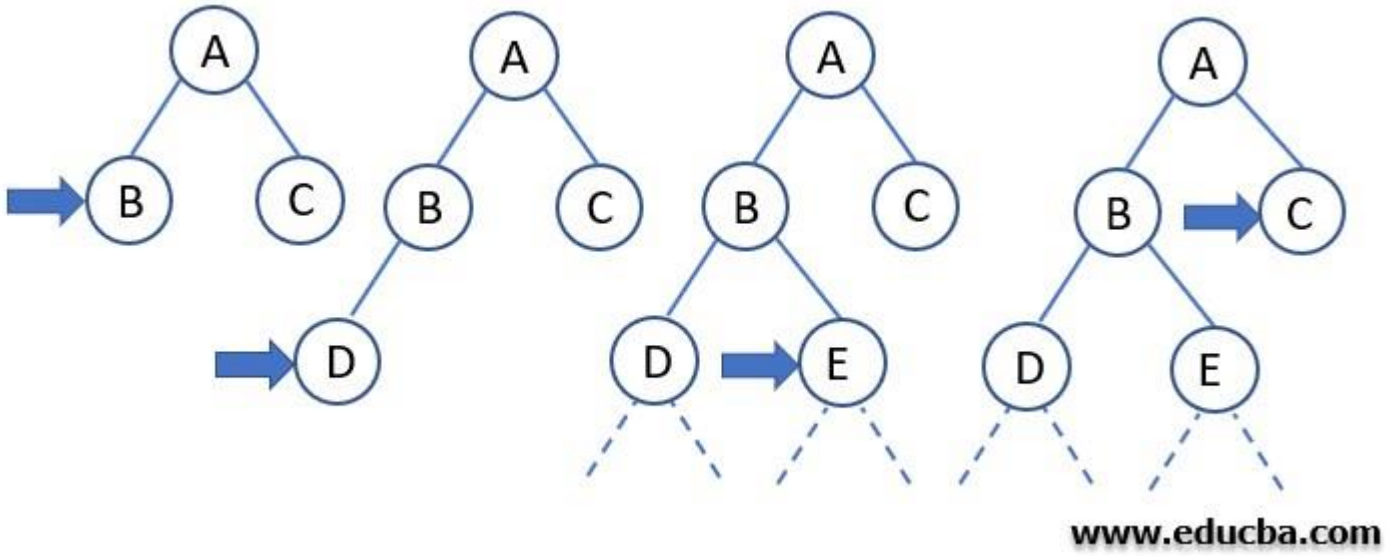
If the solution is beneath the depth bound, the search cannot find the goal (hence this search algorithm is incomplete).

- Otherwise use Iterative deepening search (IDS).



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

**In Depth Limited Search, we first set a constraint on how deep (or how far from root) will we go.**




In the above example , If we fix the depth limit to 2, DLS can be carried out similarly to the DFS until the goal node is found to exist in the search domain of the tree.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Applications

- **Real-Time Systems:** Ensures a response within a strict time or memory budget by limiting search depth.
- **Robotics:** Used for path planning where computational resources are limited, and the robot's actions are constrained to a fixed horizon.

## Advantages:

- Depth-limited search is Memory efficient.

## Disadvantages:

- Depth-limited search also has a disadvantage of incompleteness.
- It may not be optimal if the problem has more than one solution


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

The following algorithm summarises the working of depth limited search:

1. Set the depth limit to the maxdepth to search.
2. Initial node = current node
   If initial node = goal, return
3. If depth(initial node) > depth limit
   return
   else
   Expand(initial node)
   Save(successors) using stack
   Go to step 2

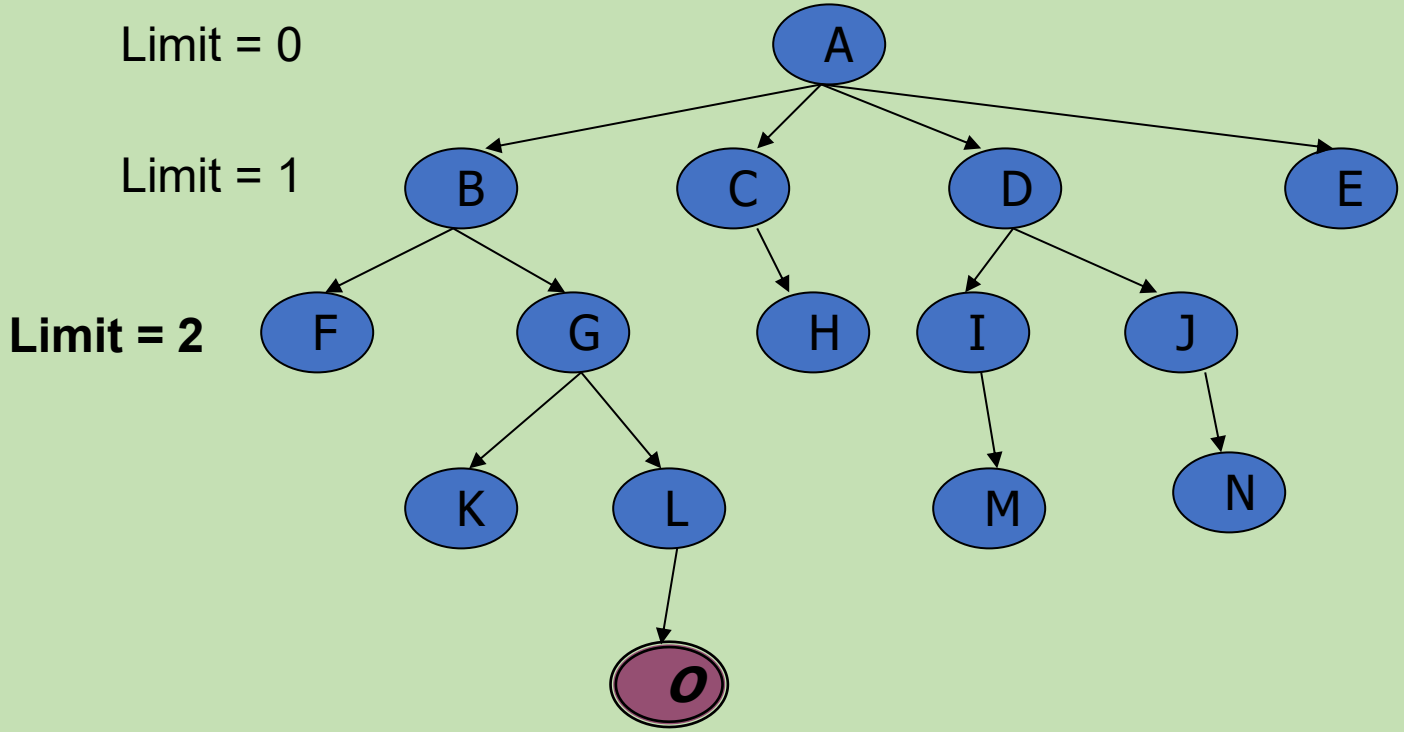
- **Completeness:** DLS search algorithm is complete if the solution is above the depth-limit.
- **Time Complexity:** Time complexity of DLS algorithm is $\mathbf{O(b^{\ell})}$.
- **Space Complexity:** Space complexity of DLS algorithm is $\mathbf{O(b \times \ell)}$.
- **Optimal:** Depth-limited search can be viewed as a special case of DFS, and it is also not optimal

even if $\ell > d$.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

**Given the following state space (tree search), give the sequence of visited nodes when using DLS (Limit = 2)**

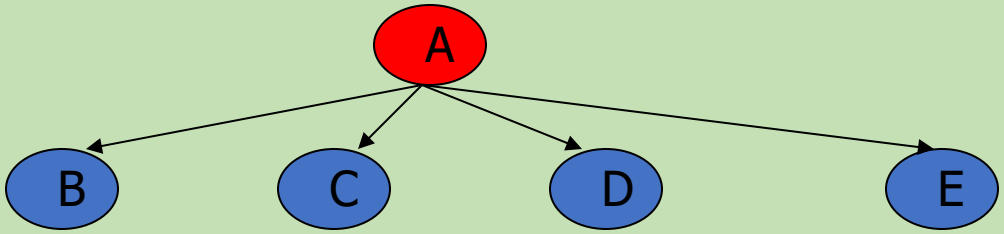





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth-Limited Search (DLS)

- A,

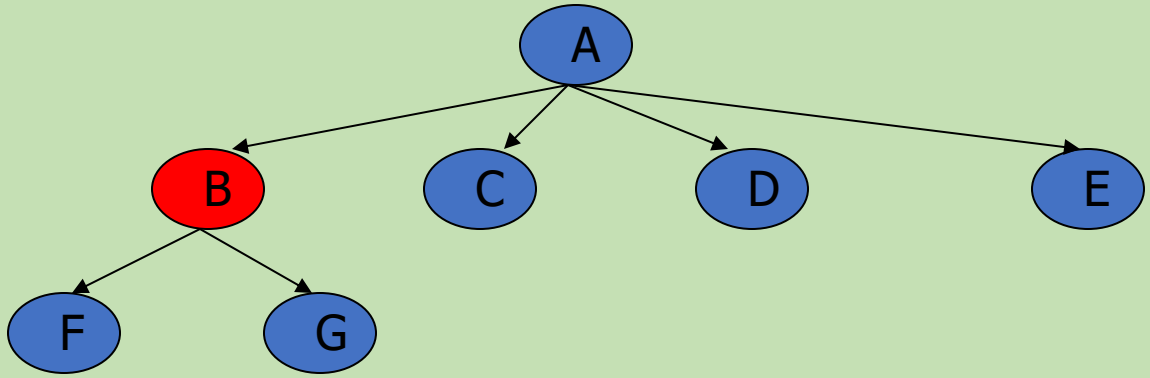



**Limit = 2**


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth-Limited Search (DLS)

- A,B,



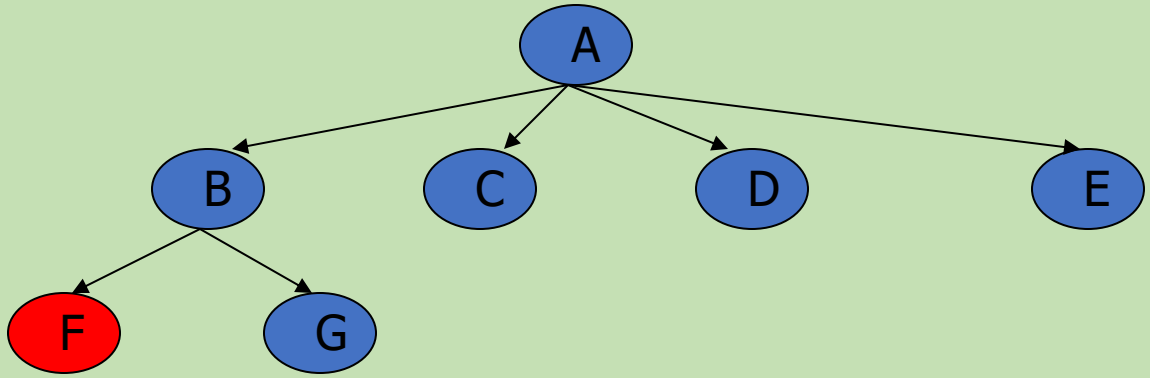

**Limit = 2**



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth-Limited Search (DLS)

- A,B,F,



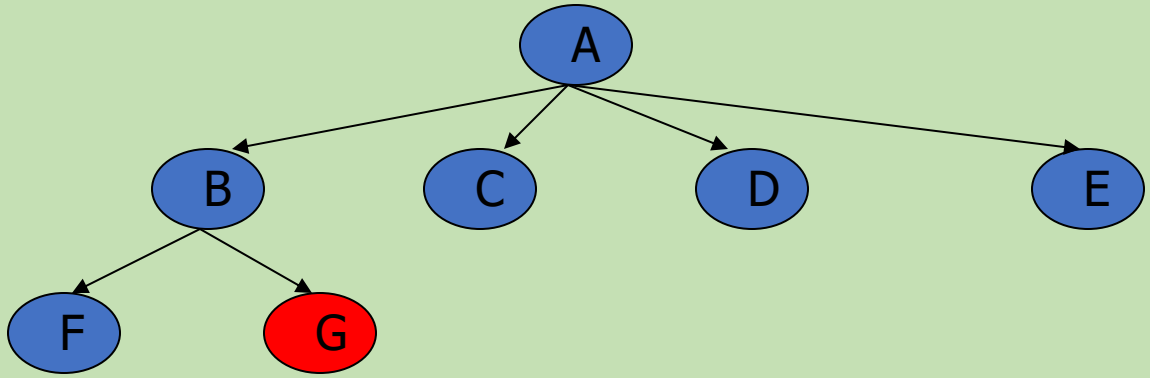

**Limit = 2**



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth-Limited Search (DLS)

- A,B,F,
- G,



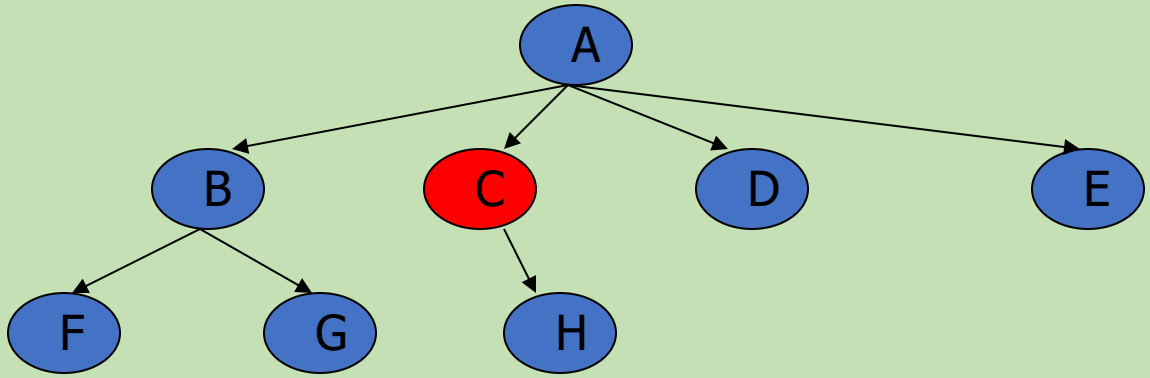

**Limit = 2**



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth-Limited Search (DLS)

- A,B,F,
- G,
- C,



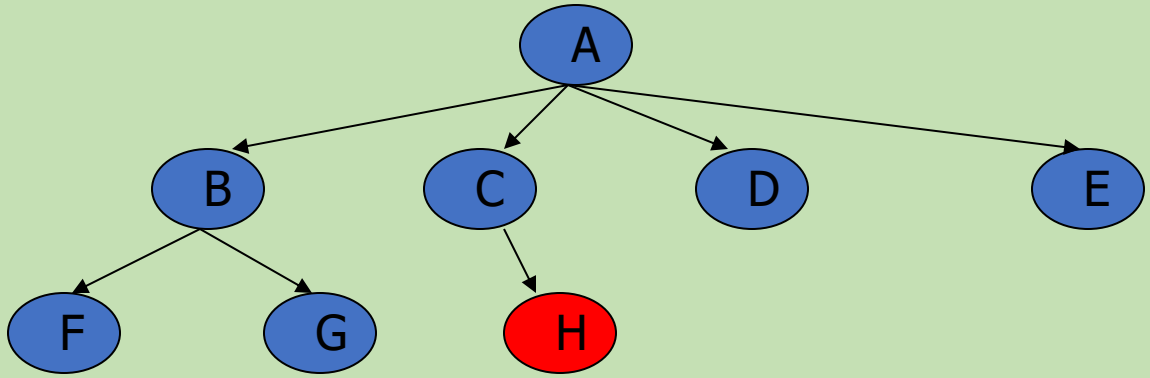

**Limit = 2**



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth-Limited Search (DLS)

- A,B,F,
- G,
- C,H,



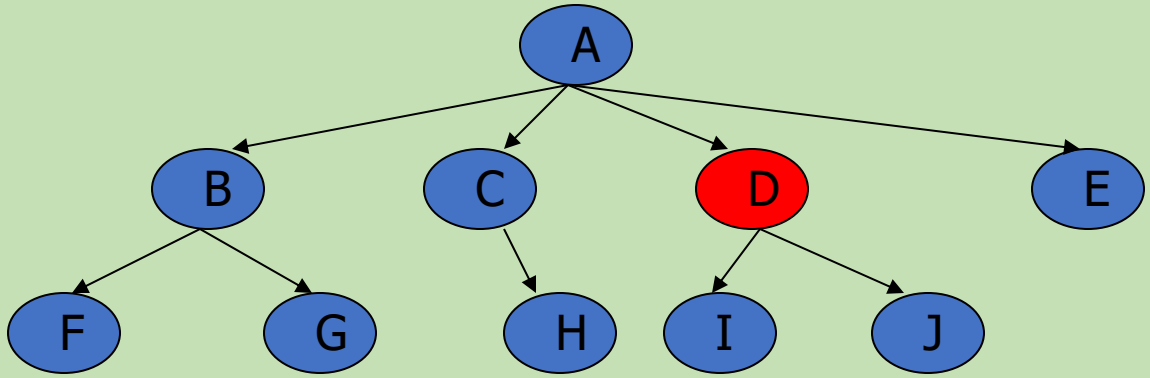

**Limit = 2**



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth-Limited Search (DLS)

- A,B,F,
- G,
- C,H,
- D,



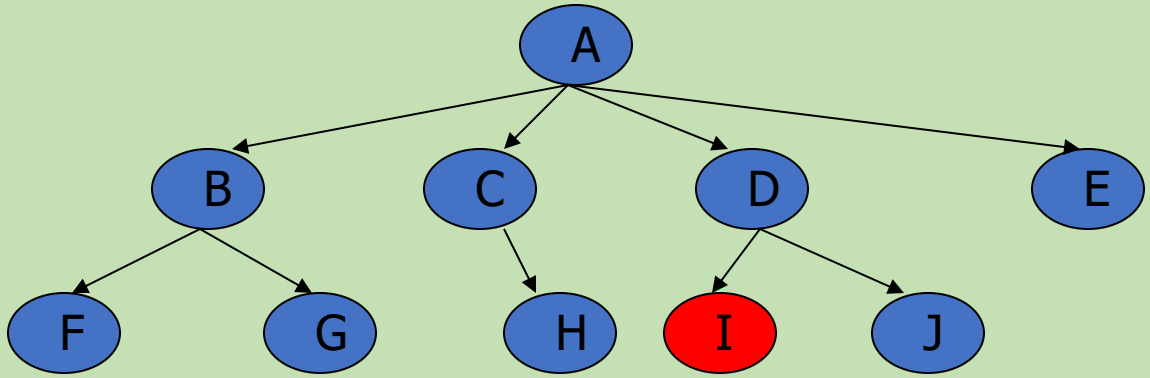

**Limit = 2**



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth-Limited Search (DLS)

- A,B,F,
- G,
- C,H,
- D,I




**Limit = 2**

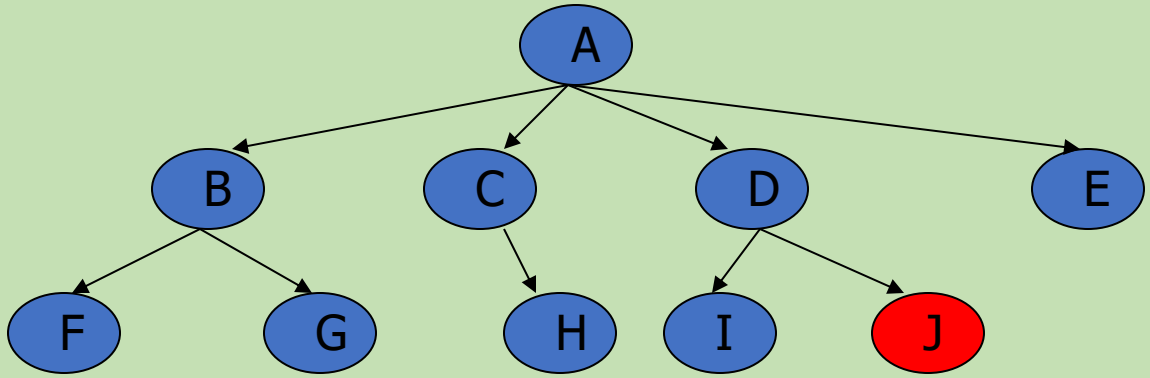


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth-Limited Search (DLS)

- A,B,F,
- G,
- C,H,
- D,I
- J,

**Limit = 2**



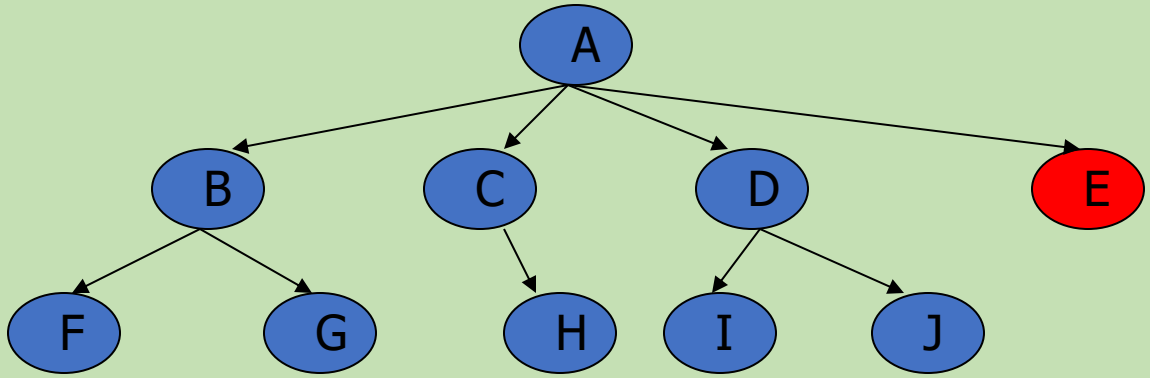



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth-Limited Search (DLS)

- A,B,F,
- G,
- C,H,
- D,I
- J,
- E

**Limit = 2**



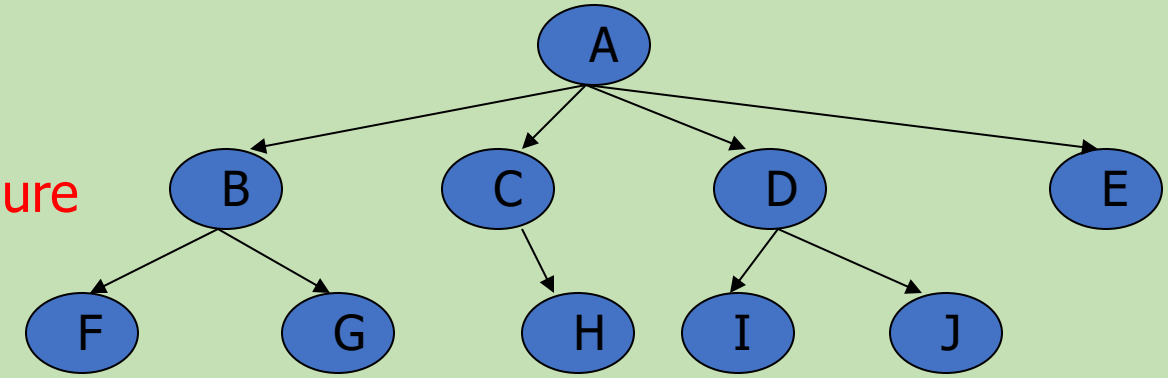



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth-Limited Search (DLS)

- A,B,F,
- G,
- C,H,
- D,I
- J,
- E, Failure

**Limit = 2**

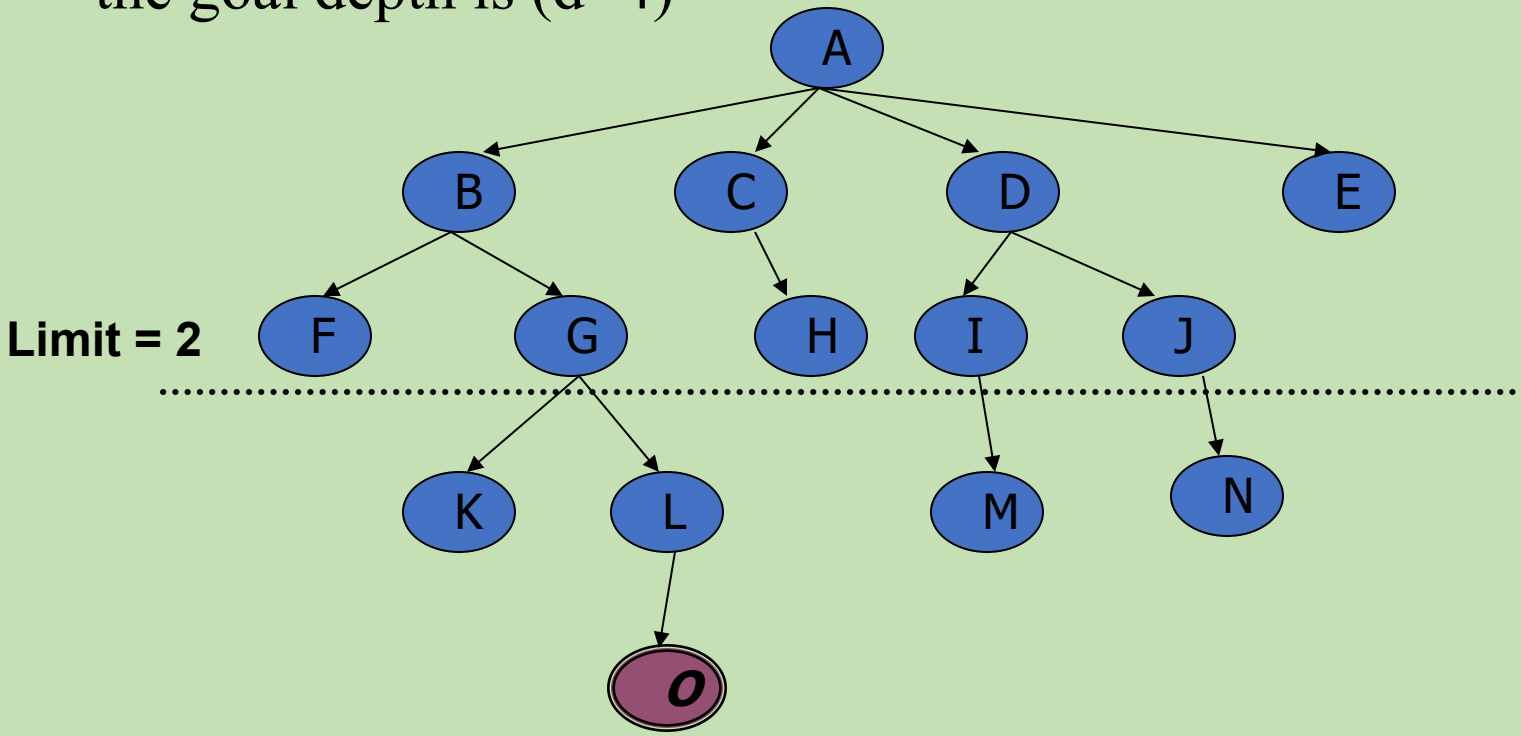





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Depth-Limited Search (DLS)

- DLS algorithm returns Failure (no solution)
- The reason is that the goal is beyond the limit (Limit =2): the goal depth is (d=4)





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Bidirectional Search Algorithm

- Bidirectional search algorithm runs two simultaneous searches, one form initial state called as forward-search and other from goal node called as backward-search, to find the goal node.
- Bidirectional search replaces one single search graph with two small sub graphs in which one starts the search from an initial vertex and other starts from goal vertex.
- The search stops when these two graphs intersect each other.
- Bidirectional search can use search techniques such as BFS, DFS, DLS, etc.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

**ADVANTAGES:**

- Bidirectional search is fast.
- Bidirectional search requires less memory.

**DISADVANTAGES:**

- Implementation of the bidirectional search tree is difficult.
- In bidirectional search, one should know the goal state in advance.

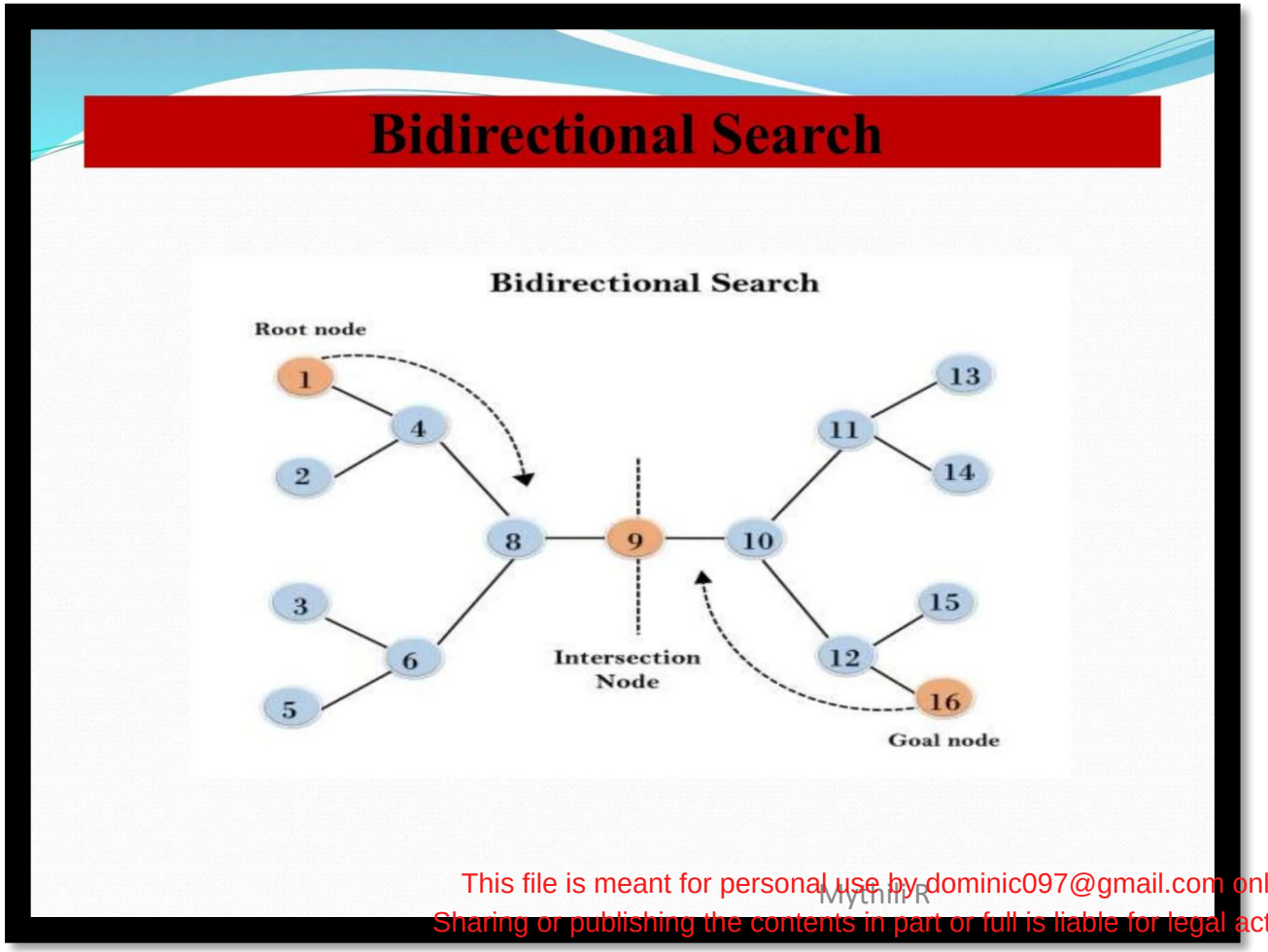


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

**Example:**

In the below search tree, bidirectional search algorithm is applied. This algorithm divides one graph/tree into two sub-graphs. It starts traversing from node 1 in the forward direction and starts from goal node 16 in the backward direction.

The algorithm terminates at node 9 where two searches meet.






This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- We have node 1 as the start/root node and node 16 as the goal node. The algorithm divides the search tree into two sub-trees.
- So, from start node 1, we do a forward search and at the same time, we do a backward search from goal node 16.
- The forward search traverse's nodes 1, 4, 8, and 9 whereas the backward search traverses through nodes 16, 12, 10, and 9.
- We see that both forward and backward search meets at node 9 called the intersection node.
- So, the total path traced by forwarding search and the path traced by backward search is the optimal solution. This is how the BS Algorithm is implemented.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# COMPARISON OF UNINFORMED SEARCH STRATEGIES

| Criterion | Breadth-First | Uniform-Cost | Depth-First | Depth-Limited | Iterative Deepening | Bidirectional (if applicable) |
|---|---|---|---|---|---|---|
| Complete? | Yes$^a$ | Yes$^{a,b}$ | No | No | Yes$^a$ | Yes$^{a,d}$ |
| Time | $O(b^d)$ | $O(b^{1+\lfloor C^*/\epsilon \rfloor})$ | $O(b^m)$ | $O(b^\ell)$ | $O(b^d)$ | $O(b^{d/2})$ |
| Space | $O(b^d)$ | $O(b^{1+\lfloor C^*/\epsilon \rfloor})$ | $O(bm)$ | $O(b\ell)$ | $O(bd)$ | $O(b^{d/2})$ |
| Optimal? | Yes$^c$ | Yes | No | No | Yes$^c$ | Yes$^{c,d}$ |

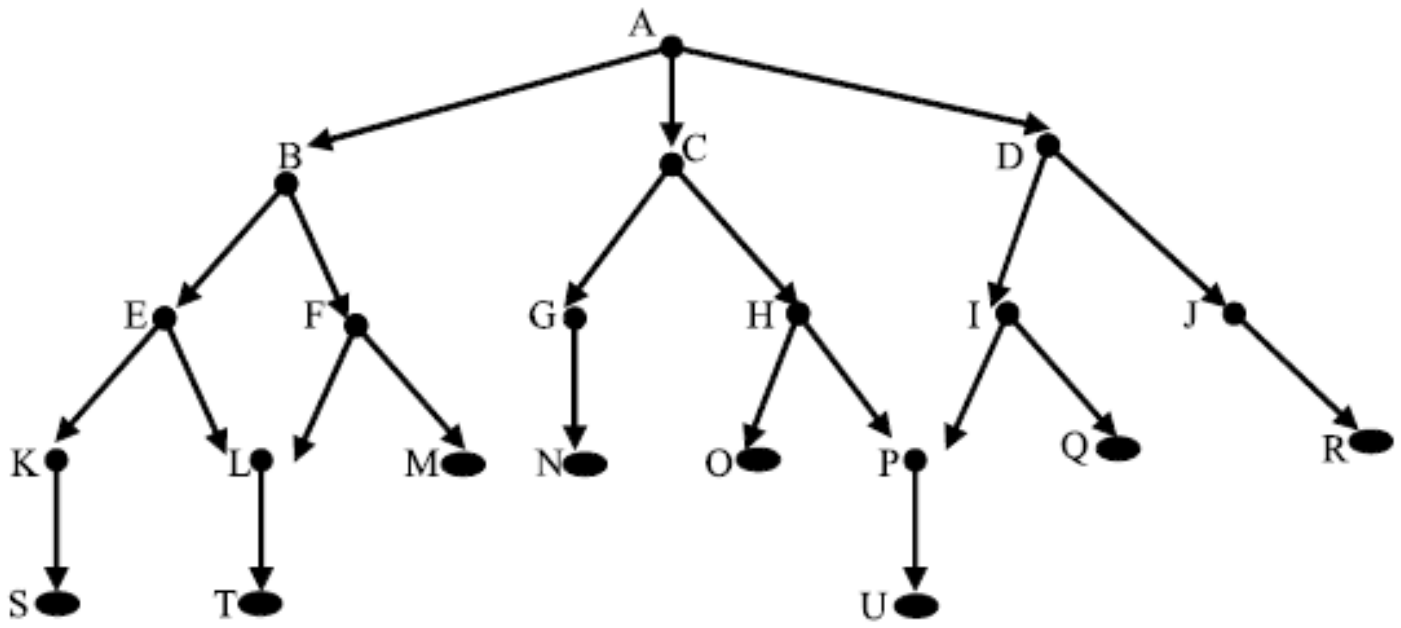
**Figure 3.21** Evaluation of tree-search strategies. $b$ is the branching factor; $d$ is the depth of the shallowest solution; $m$ is the maximum depth of the search tree; $l$ is the depth limit. Superscript caveats are as follows: $^a$ complete if $b$ is finite; $^b$ complete if step costs $\geq \epsilon$ for positive $\epsilon$; $^c$ optimal if step costs are all identical; $^d$ if both directions use breadth-first search.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# SOLVE THIS

Q.2 Apply BFS algorithm on the following tree (M is goal node)






This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# ***Informed Search/Heuristic Search***

- Heuristic Search Uses domain-dependent (heuristic) information beyond the definition of the problem itself in order to search the space more efficiently.
- The following are some ways of using heuristic information:
  - Deciding which node to expand next, instead of doing the expansion in a strictly breadth-first or depth-first order;
  - In the course of expanding a node, deciding which successor or successors to generate, instead of blindly generating all possible successors at one time:
  - Deciding that certain nodes should be discarded, or pruned, from the search space.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- Heuristics is a guess work, or additional information about the problem. It may miss the solution, if wrong heuristics is supplied. However, in almost all problems with correct heuristic information, it provides good solution in reasonable time
- Informed search can solve much complex problem which could not be solved in another way.
- A heuristic function at a node n is an estimate of the optimum cost from the current node toa goal. Denoted by h(n)
- h(n) = estimated cost of the cheapest path from node n to a goal node
- For example, suppose you want to find a shortest path from Kolkata to Guwahati, then heuristic for Guwahati may be straight-line distance between Kolkata and Guwahati, that is
- h(Kolkata) = euclideanDistance(Kolkata,Guwahati)
- The following are the informed search algorithm:
  1. Greedy Best-First Search
  2. A* algorithm
  3. AO* algorithm


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# ***Greedy Best-First search***

- Greedy best-first search algorithm always selects the path which appears best at that moment. It is the combination of depth-first search and breadth-first search algorithms.
- It uses the heuristic function and search. Best-first search allows us to take the advantages of both algorithms.
- With the help of best-first search, at each step, we can choose the most promising node.
- In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e. f(n)= g(n) where h(n)= estimated cost from node n to the goal.
- The greedy best first algorithm is implemented by the **priority queue** (or to store the heuristic function value)


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

**Best first search algorithm:**

1. **Initialize:** Set OPEN={s} , CLOSED={ };

   $f(s)=h(s)$

2. **Fail:** If OPEN={ }, terminate with failure.
3. **Select:** Select the minimum cast state a from OPEN, save n in CLOSED.
4. **Terminate:** If $a \in$ Goal node, terminate with success and return f(n), else
5. **Expend:** For each successor, m of n,

   If $m \notin$ [OPEN$\cup$CLOSED]

   Set f(m)=h(m) and Insert m in OPEN

   .....

6. **LOOP:** Goto Step 2



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

OR Pseudocode (for Best-First Search algorithm)

**Step 1:** Place the starting node into the OPEN list.

**Step 2:** If the OPEN list is empty, Stop and return failure.

**Step 3:** Remove the node n, from the OPEN list which has the lowest value of h(n), and places it in the CLOSED list.

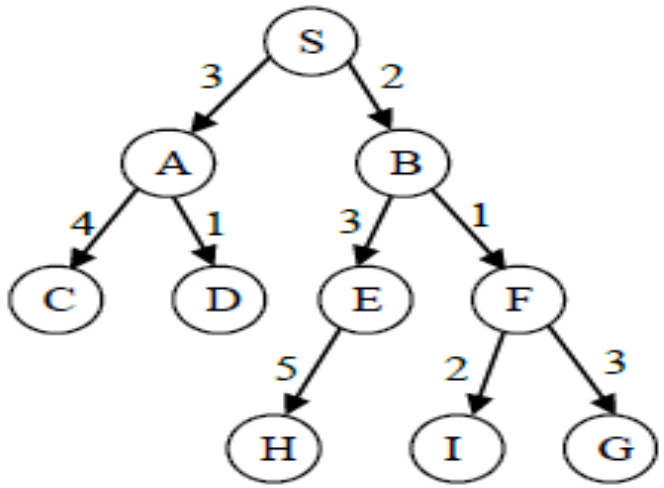
**Step 4:** Expand the node n, and generate the successors of node n.

**Step 5:** Check each successor of node n, and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.

**Step 6:** For each successor node, algorithm checks for evaluation function f(n), and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both lists, then add it to the OPEN list.

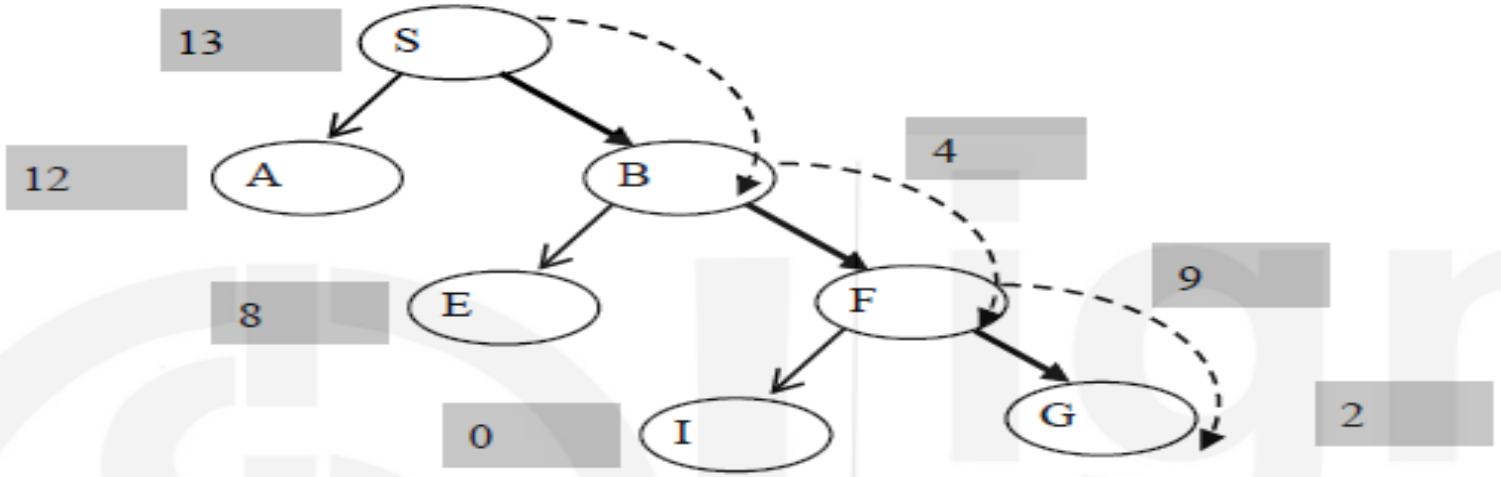
**Step 7:** Return to Step 2.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.




| node | h(n) |
|---|---|
| A | 12 |
| B | 4 |
| C | 7 |
| D | 3 |
| E | 8 |
| F | 2 |
| H | 4 |
| I | 9 |
| S | 13 |

Here, we are using two lists which are **OPEN** and **CLOSED** Lists. Following are the iteration for traversing the above example.





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- **Initialization:** Open [A, B], Closed [S]
- **Iteration 1 :** Open [A], Closed [S, B]
- **Iteration 2 :** Open [E, F, A], Closed [S, B] : Open [E, A], Closed [S, B, F]
- **Iteration 3 :** Open [I, G, E, A], Closed [S, B, F] : Open [I, E, A], Closed [S, B, F, G]
- Hence the final solution path will be **S----> B----->F----> G**

**Evaluation of Best-First Search algorithm:**

- **Time Complexity:**

The worst-case time complexity of Greedy best first search is O(bm).

- **Space Complexity:**

The worst-case space complexity of Greedy best first search is O(bm). Where, m is the maximum depth of the search space.

- **Complete:** Greedy best-first search is also incomplete, even if the given state space is finite.
- **Optimal:** Greedy best first search algorithm is not optimal.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

**Example3:** Apply Greedy Best-First Search algorithm on the following graph (L is a goal node).

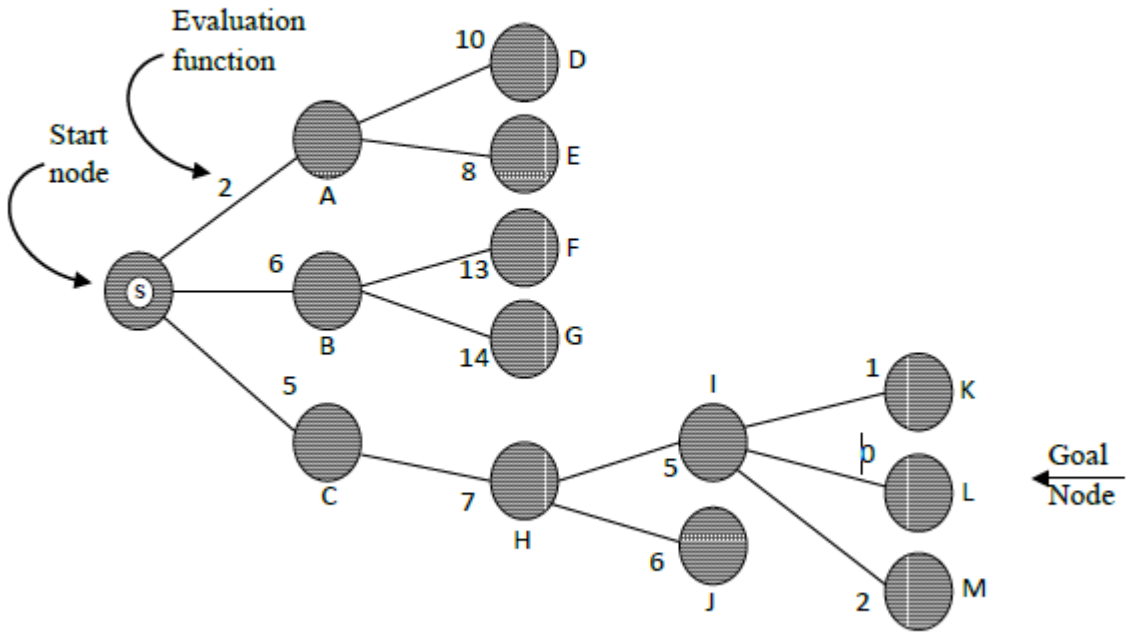



Fig. 2.17: Greedy best first search graph.

| Step | Node being expanded | Children (on expansion) | Available nodes (to search) | Node Chosen |
|---|---|---|---|---|
| 1. | S | (A:2), (B:6), (C:5) | (A:2), (B:6), (C:5) | (A:2) |
| 2. | A | (D:10), (E:8) | (B:6), (C:5), (D:10), (E:8) | (C:5) |
| 3. | C | (H:7) | (B:6), (D:10), (E:8), (H:7) | (B:6) |
| 4. | B | (F:13), (G:14) | (D:10), (E:8), (H:7), (F:13), (G:14) | (H:7) |
| 5. | H | (I:5), (J:6) | (D:10), (E:8), (F:13), (G:14), (I:5), (J:6) | (I:5) |
| 6. | I | (K:1), (L:0), (M:2) | (D:10), (E:8), (H:7), (F:13), (G:14), (J:6), (K:1), (L:0), (M:2) | Goal node is found. So, search stops now |



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# ***A* Algorithm***

- A* search is the most commonly known form of best-first search.
- It uses heuristic function h(n), and cost to reach the node n from the start state g(n). It has combined features of uniform cost search (UCS) and greedy best-first search, by which it solves the problem efficiently.
- A* search algorithm finds the shortest path through the search space using the heuristic function.
- This search algorithm expands less search tree and provides optimal result faster.
- In A* search algorithm, we use search heuristic as well as the cost to reach the node.
- Hence, we can combine both costs as following, and this sum is called as a **fitness number (Evaluation Function).**


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

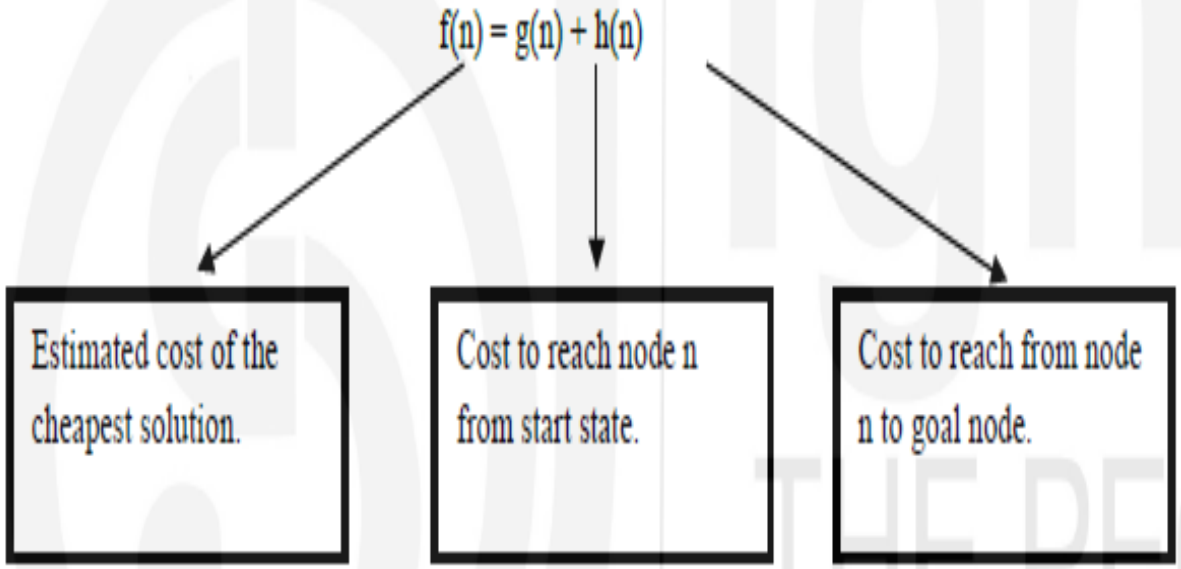



Fig 9 Evaluation function f(n) in A* Algorithm

A* algorithm evaluation **function** $f(n)$ is defined as $f(n)=g(n)+h(n)$

Where $g(n)$ = sum of edge costs from start state to n

And $h(n)$ = estimate of lowest cost path from node n→goal node.

# Algorithm A*

1. Initialize:

Set OPEN = (s), CLOSED = {}, g(s) = 0,

f(s) = h(s)

2. Fail: If OPEN = { }, Terminate & fail

3. Select : Select the minimum cost state, n, from OPEN, Save n in CLOSED.

4. Terminate: If n ∈ G, terminate with success, and return f(n)

5. Expand: For each successor, m, of n

If m ∉ [OPEN ∪ CLOSED]

Set g(m) = g(n) + C(n,m)

Set f(m) = g(m) +h(m)

Insert m in OPEN

If m ∈ [OPEN ∪ CLOSED]

Set g(m) = min {min {g(m), g(n) + C(n,m)}

Set f(m) = g(m) + h(m)

If f(m) has decreased and m ∈ CLOSED, move m to OPEN



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Pseudocode

**Step-1:** Define a list OPEN. Initially, OPEN consists of a single node, the start node S.

**Step-2:** If the list is empty, return failure and exit.

**Step-3:** Remove node n with the smallest value of f(n) from OPEN and move it to list CLOSED.

If node n is a goal state, return success and exit.

**Step-4:** Expand node n.

**Step-5:** If any successor to n is the goal node, return success and the solution by tracing the path from goal node to S.

Otherwise, go to Setp-6.

**Step-6:** For each successor node,

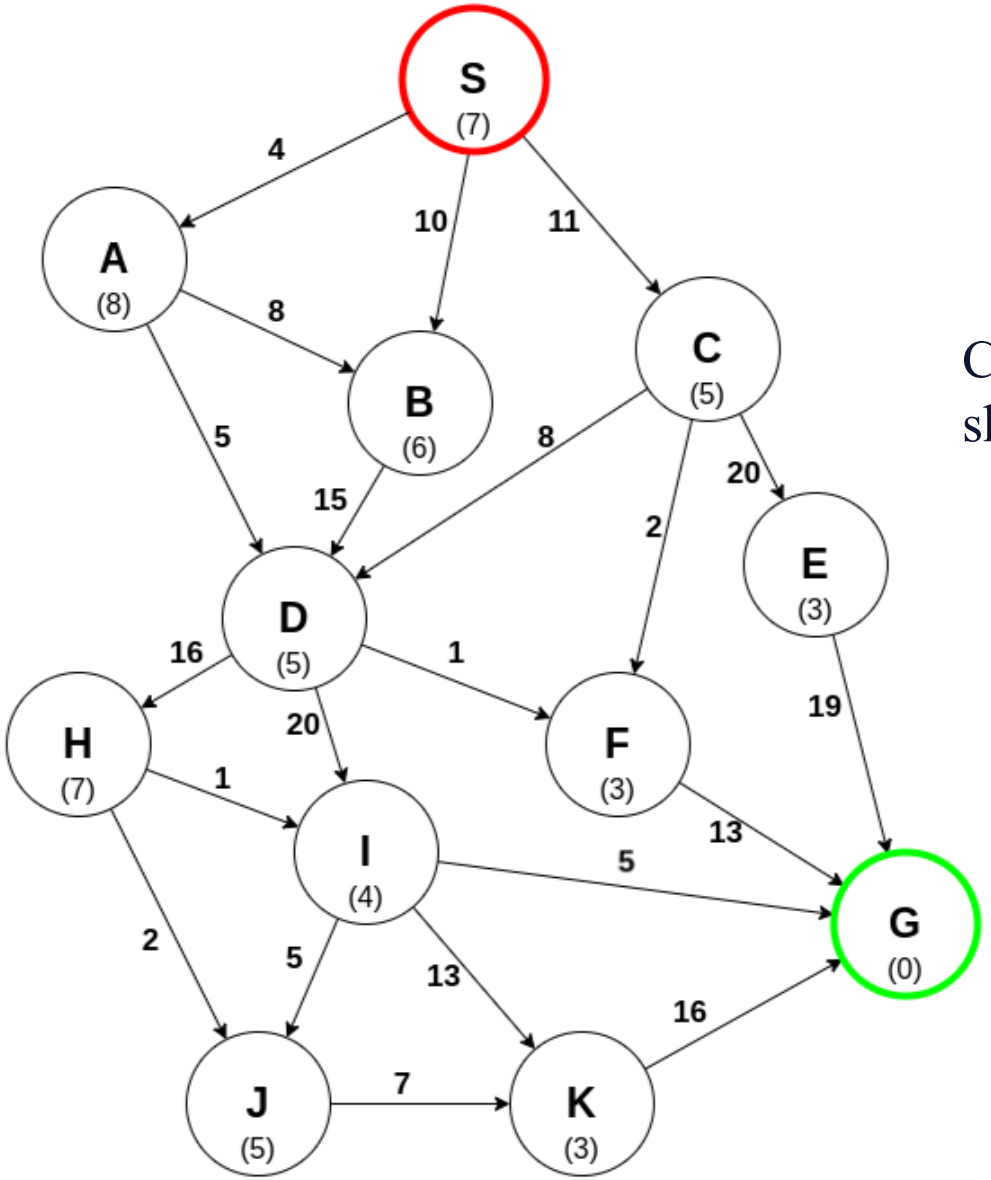
Apply the evaluation function f to the node.

If the node has not been in either list, add it to OPEN.

**Step-7:** Go back to Step-2.



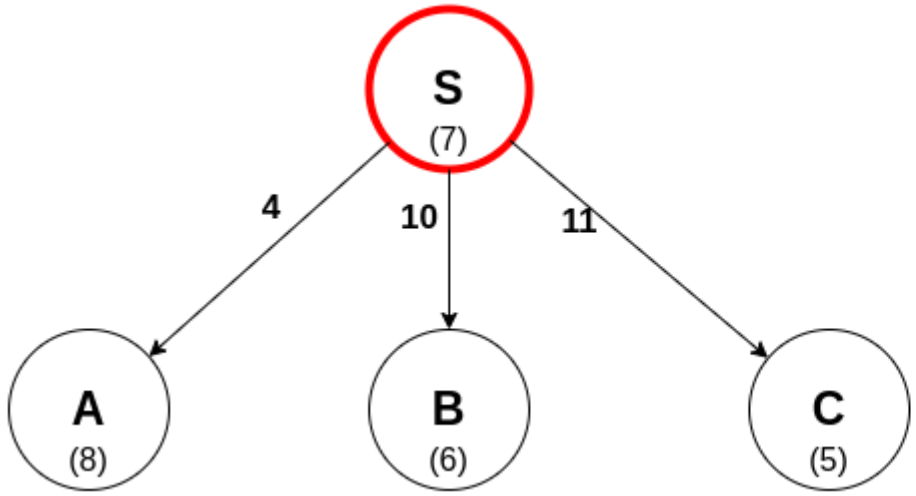
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.




Consider the following example of trying to find the shortest path from **S** to **G** in the following graph


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- Each edge has an associated weight, and each node has a heuristic cost (in parentheses).
- An open list is maintained in which the node **S** is the only node in the list. The search tree can now be constructed.
- Exploring **S**

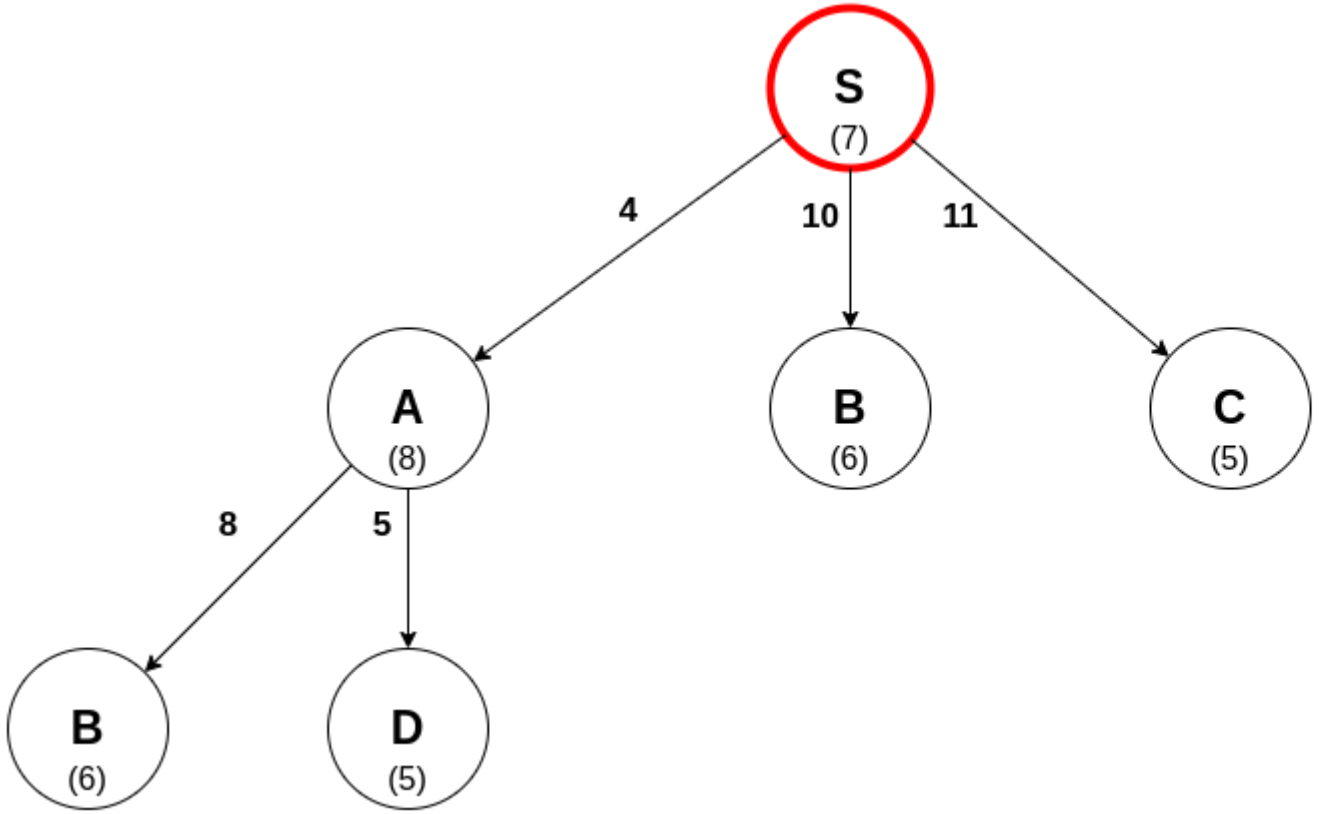



| Node[cost] |
| --- |
| A[12] |
| B[16] |
| C[16] |

| Closed List |
| --- |
| S |


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

**A** is the current most promising path, so it is explored next:

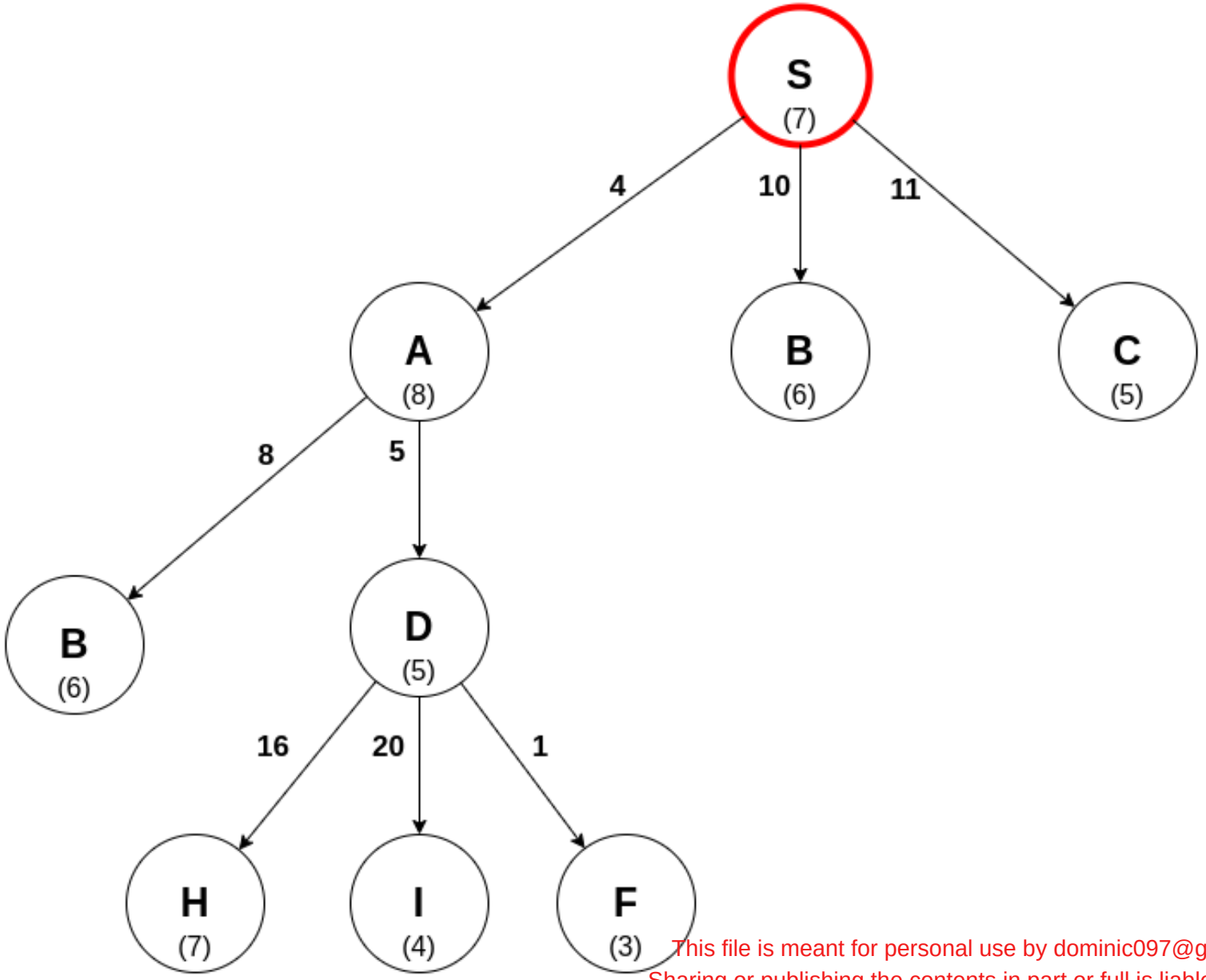



| Node[cost] |
|---|
| D[14] |
| C[16] |
| B[16] |
| B[18] |

| Closed List |
|---|
| S |
| A |


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Exploring **D**:

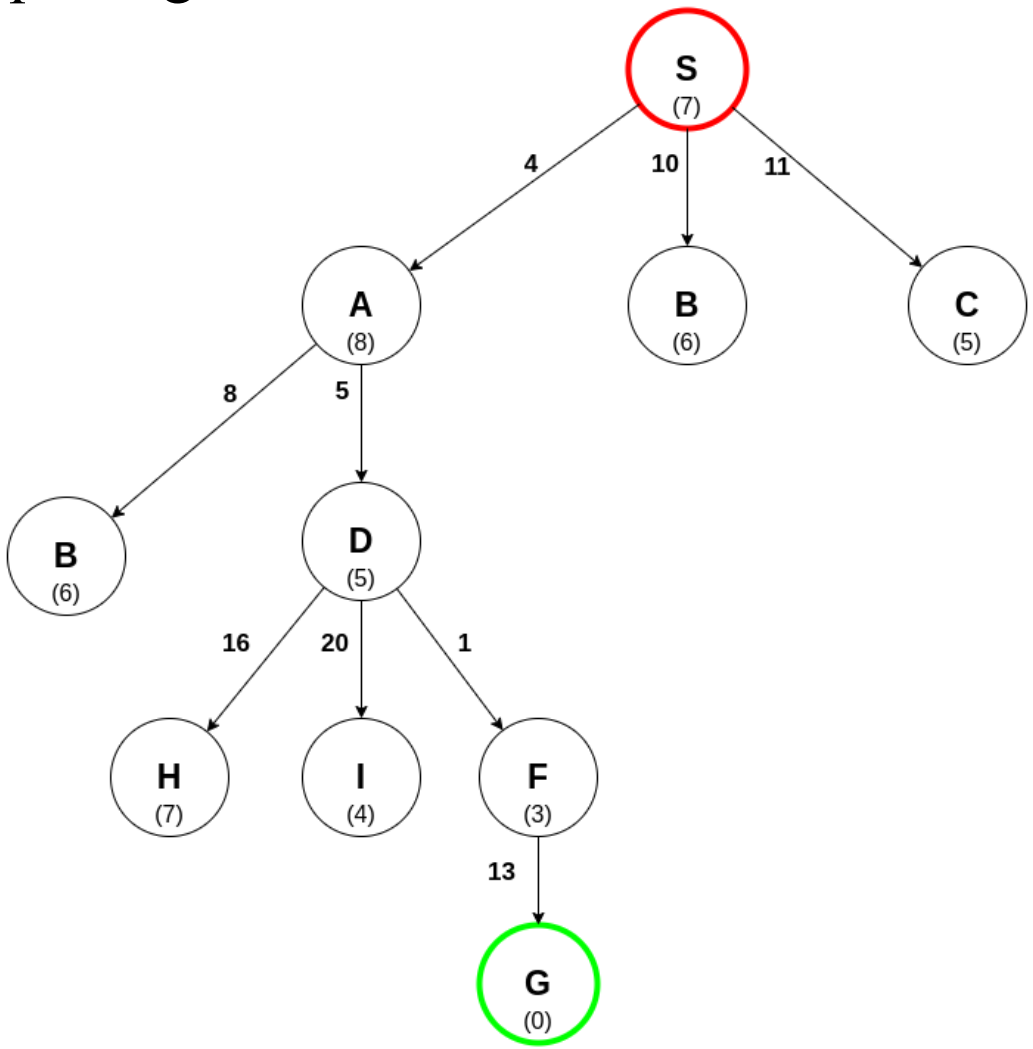



| Node[cost] |
|---|
| F[13] |
| C[16] |
| B[16] |
| H[32] |
| I[33] |
| B[18] |

| Closed List |
|---|
| S |
| A |
| D |

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Exploring F

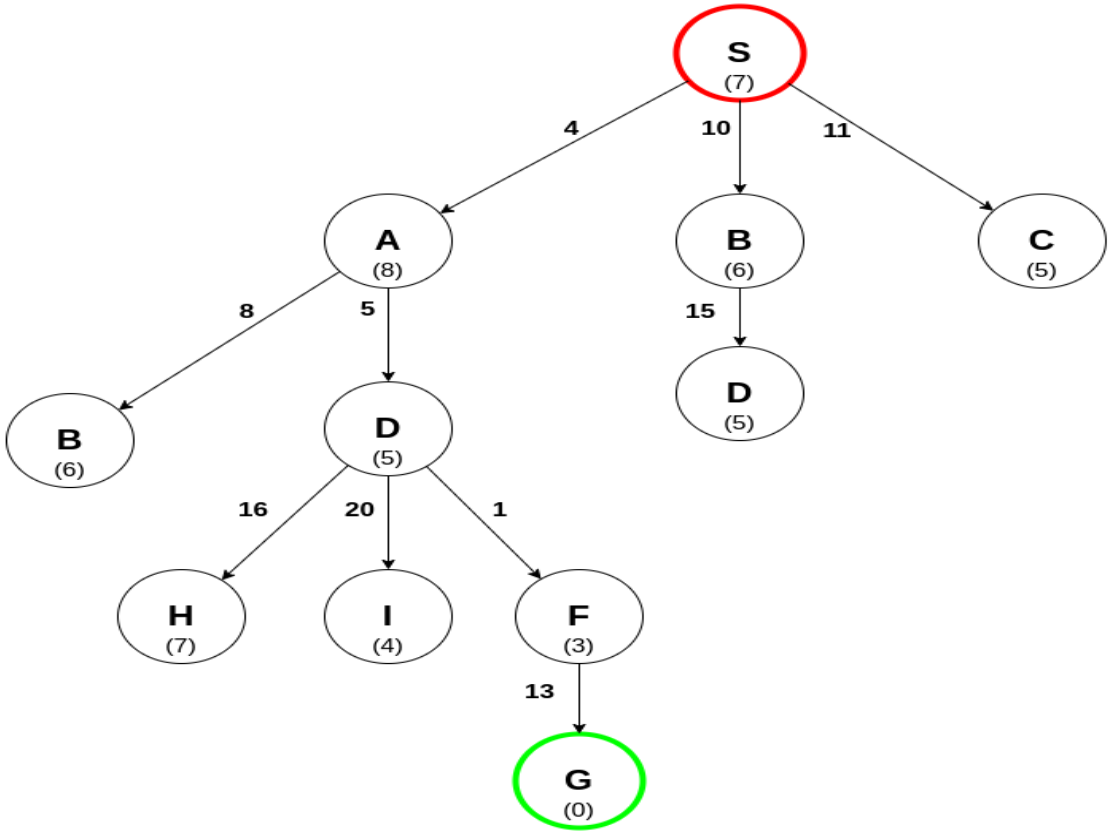



| Node[cost] |
|---|
| B[16] |
| C[16] |
| B[18] |
| H[32] |
| I[33] |
| G[23] |

| Closed List |
|---|
| S |
| A |
| D |
| F |


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Notice that the goal node **G** has been found. However, it hasn't been explored, so the algorithm continues because there may be a shorter path to G. The node **B** has two entries in the open list: one at a cost of 16 (child of **S**) and one at a cost of 18 (child of **A**). The one with the lowest cost is explored next:

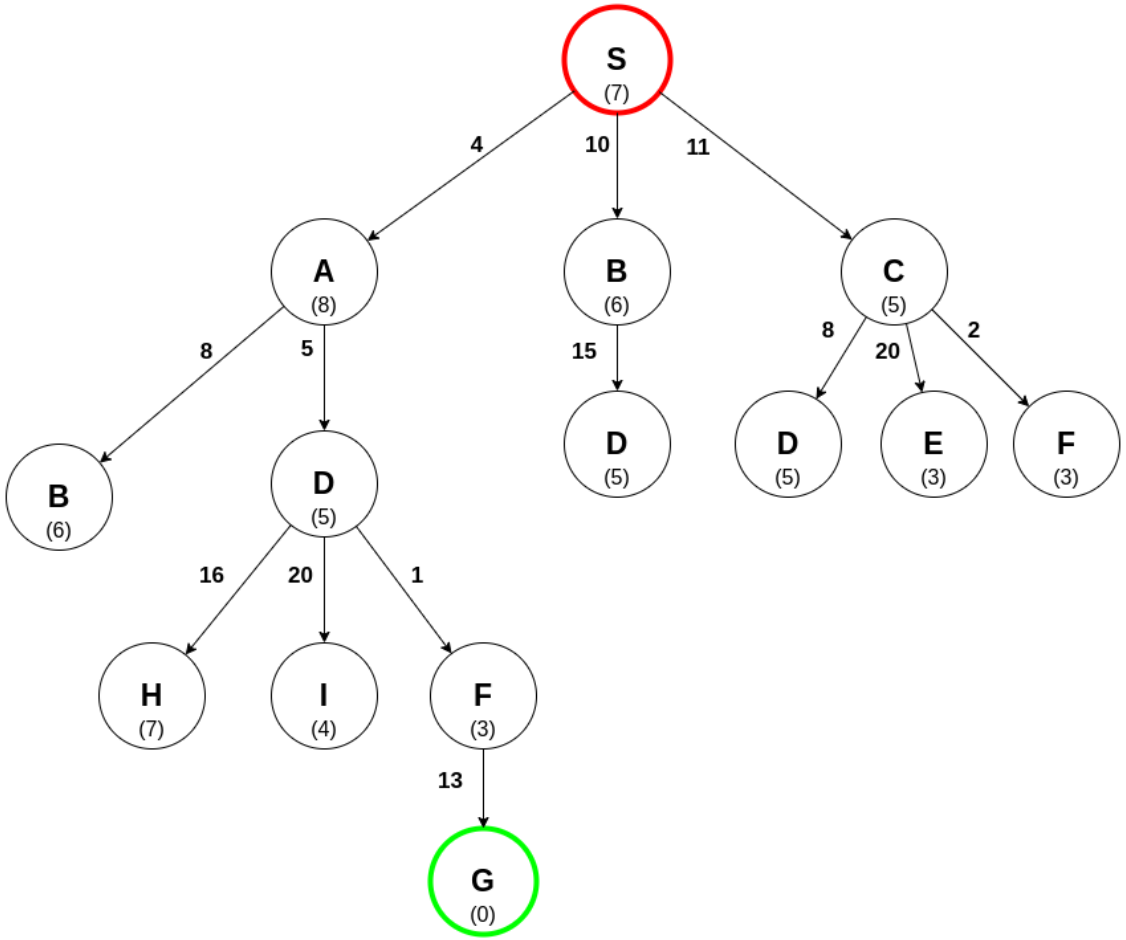



| Node[cost] |
|---|
| C[16] |
| G[23] |
| B[18] |
| H[32] |
| I[33] |

| Closed List |
|---|
| S |
| A |
| D |
| F |
| B |


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# Exploring C

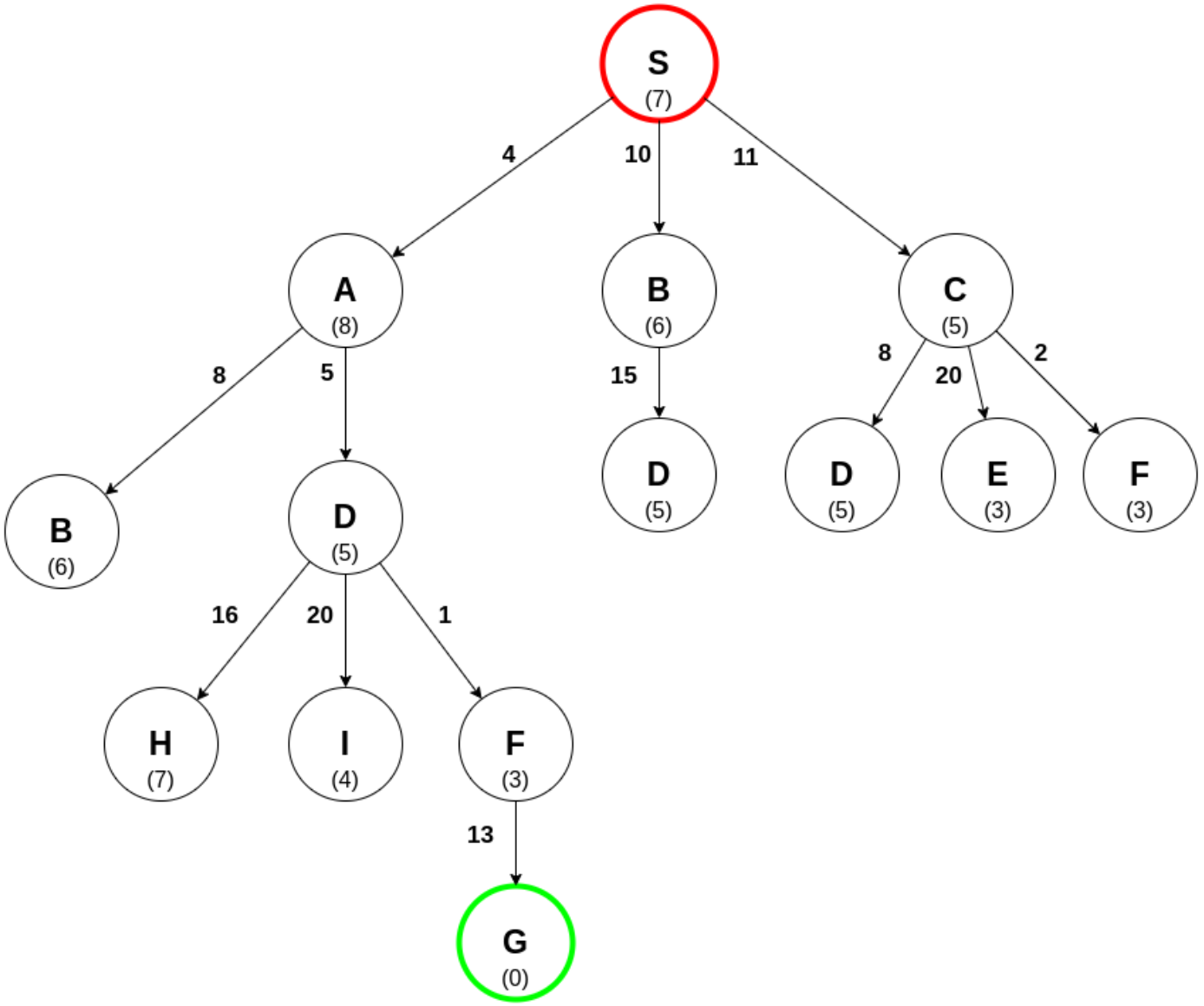



| Node[cost] |
|---|
| B[18] |
| G[23] |
| I[33] |
| H[32] |
| E[36] |

| Closed List |
|---|
| S |
| A |
| D |
| F |
| B |
| C |



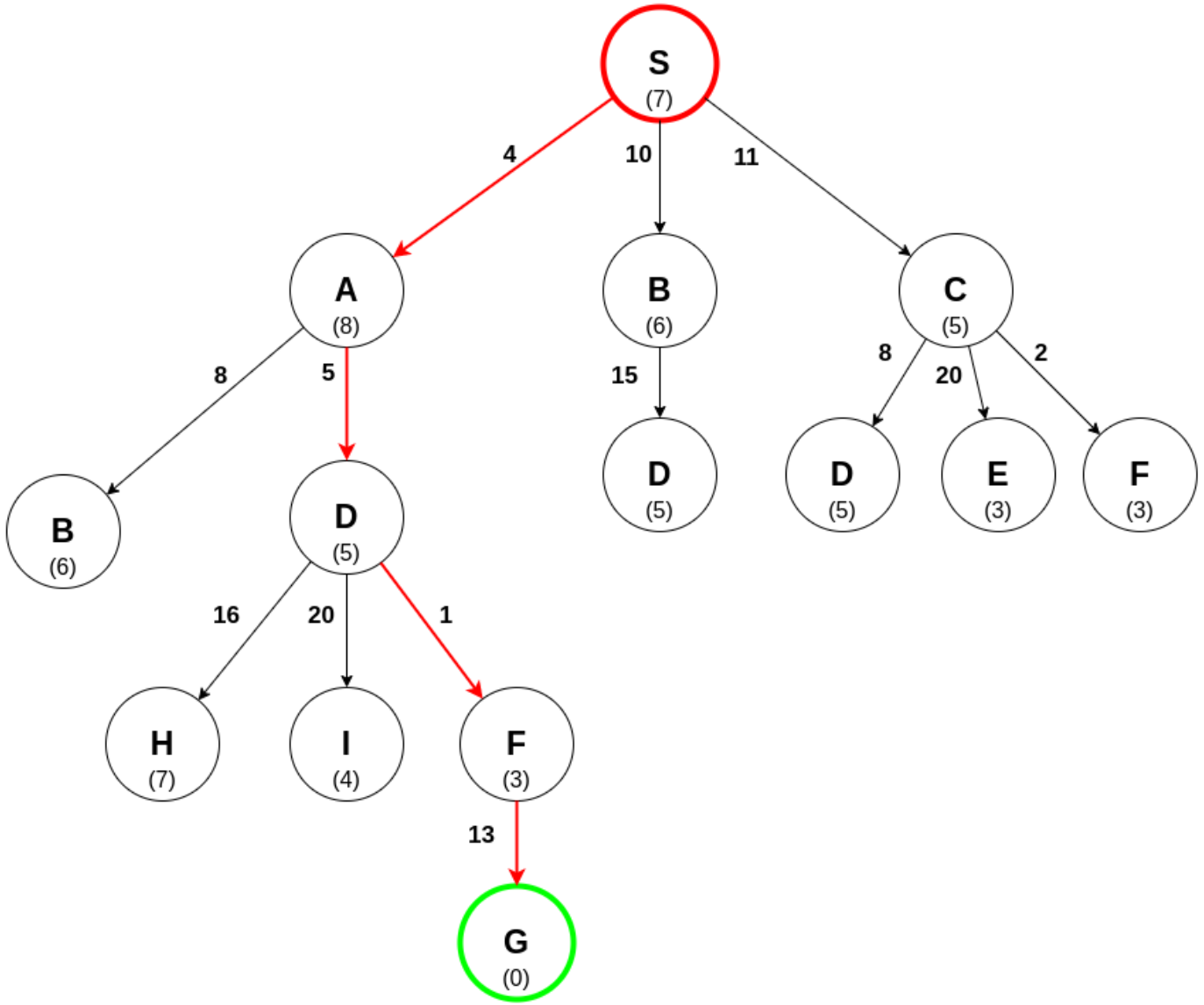
This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.




| Node[cost] |
|---|
| G[23] |
| I[33] |
| H[32] |
| E[36] |

| Closed List |
|---|
| S |
| A |
| D |
| F |
| B |
| C |


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.




| Node[cost] |
|---|
| I[33] |
| H[32] |
| E[36] |

| Closed List |
|---|
| S |
| A |
| D |
| F |
| B |
| C |
| G |

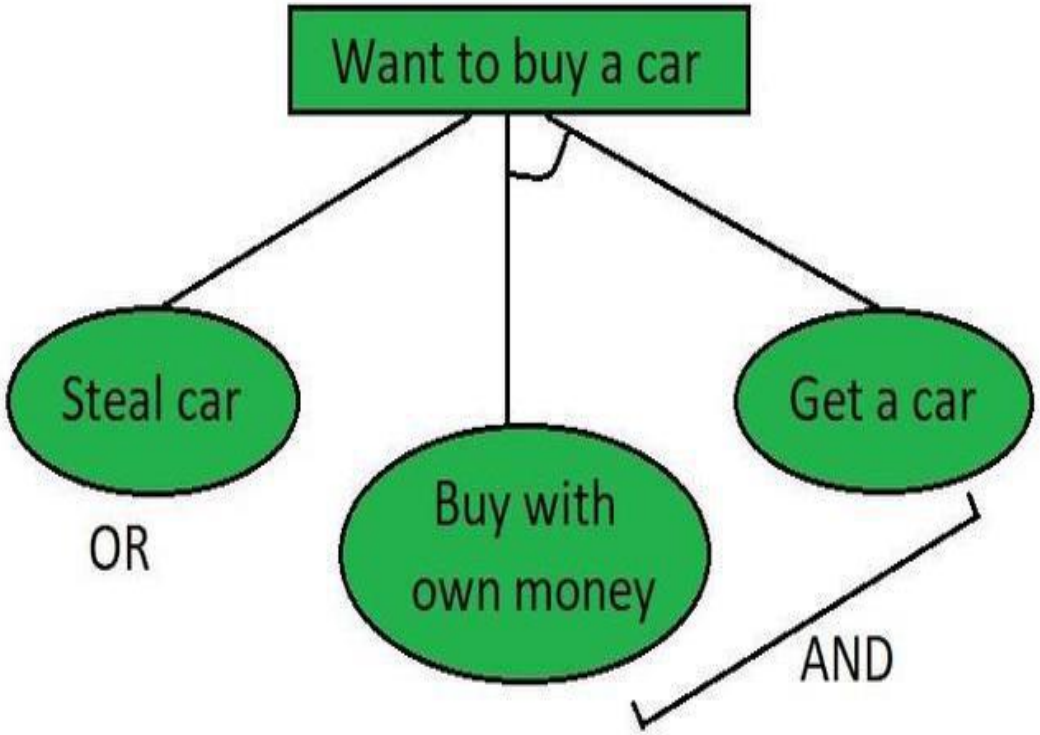

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# ***AO* Algorithm***

- The ***AO algorithm**** **(short for "And-Or Star")** is a powerful **best-first search** method used to solve problems that can be represented as a **directed acyclic graph (DAG)**.
- Unlike traditional algorithms like **A***, which explore a single path, the AO* algorithm evaluates multiple paths simultaneously.
- This makes it more efficient for problems that involve **AND-OR nodes**.
- The AND nodes represent tasks where all child nodes must be satisfied, while OR nodes offer multiple alternative paths where only one child node needs to be satisfied to achieve the goal.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

Consider the process of buying a car. This process can be broken down into smaller tasks using an **AND-OR graph**. The AND section might include securing funds or applying for a loan, while the OR section could present different options for acquiring the car, such as purchasing with cash or financing. Each subproblem in the AND-OR graph must be solved before the main issue is resolved.




- The AO* algorithm uses **knowledge-based search**, where both the start and target states are predefined. By leveraging heuristics, it identifies the best path, significantly reducing time complexity. Compared to the *A algorithm**, AO* is more efficient when solving problems represented as **AND-OR graphs**, as it evaluates multiple paths at once.
- This combination of **best-first search**, **DAG representation**, and **heuristic optimization** makes the *AO algorithm** ideal for tackling complex problems that can be divided into interdependent tasks. Incorporating these techniques ensures faster and more effective problem-solving.

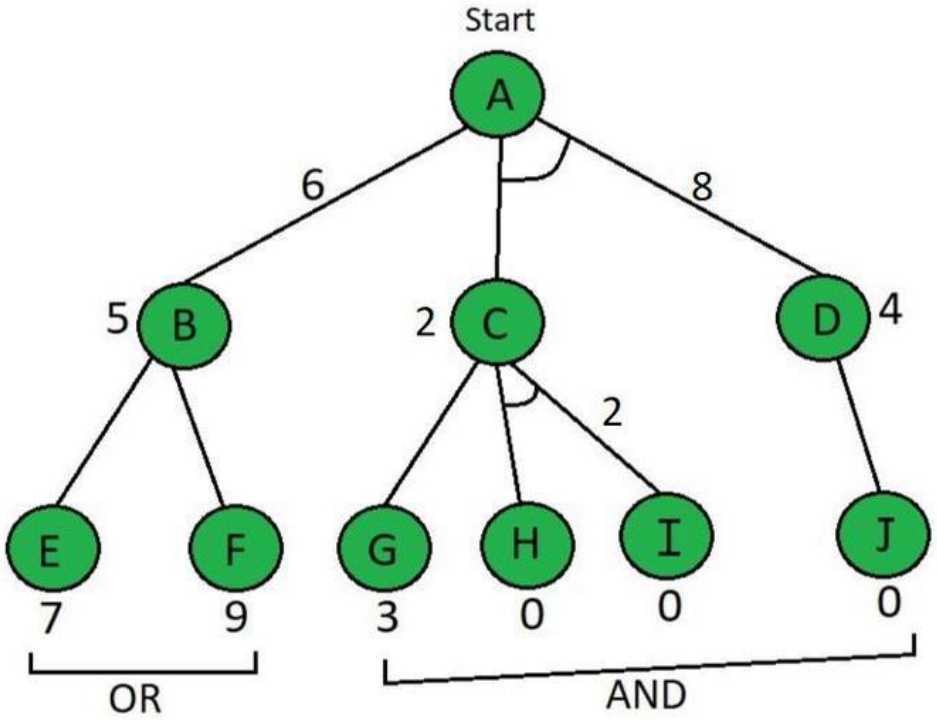

This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# ***Steps of AO****

- Start from the initial node.
- Expand nodes using a **heuristic function (h(n))** that estimates the cost.
- For an **OR node**:
  - Choose the child with the **lowest estimated cost**.
- For an **AND node**:
  - Expand **all children** (since all subproblems must be solved).
  - The cost is the **sum of the children's costs**.
- Update parent nodes with new best estimates.
- Continue expanding the most promising paths until a solution graph (covering all required subgoals) is found.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

How the *AO algorithm** works using an **AND-OR graph**. Each node in the graph is assigned a heuristic value, denoted as **h(n)**, and the edge length is considered as **1.**




**Step 1: Initial Evaluation Using f(n) = g(n) + h(n)**

- Starting from node **A**, we use the evaluation function

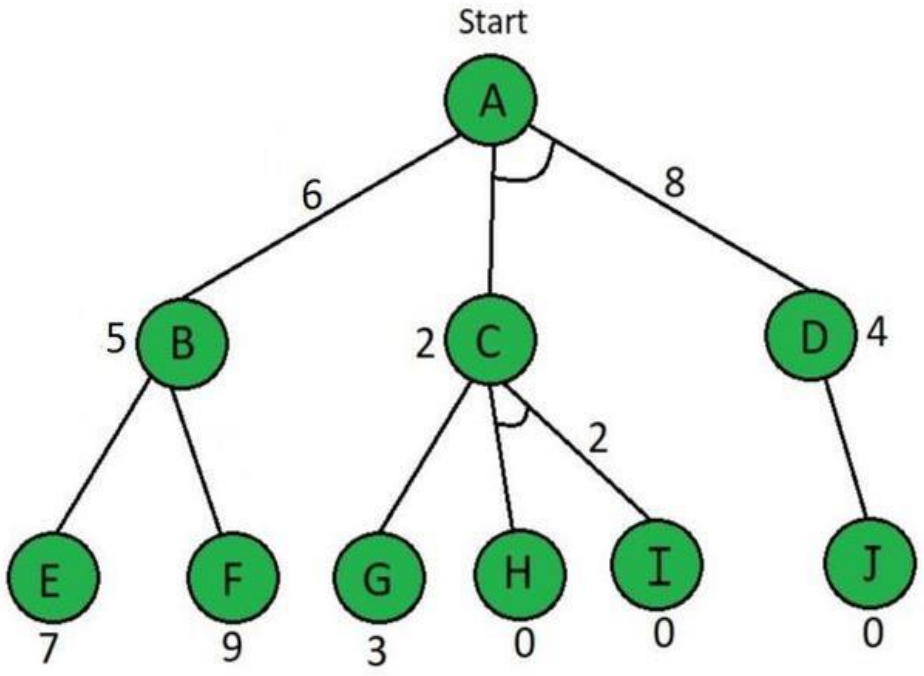
f(A -> B) = g(B) + h(B)
= 1 + 5 (g(n) = 1 is the default path cost)
= 6

For the path involving AND nodes (C and D):

f(A -> C + D) = g(C) + h(C) + g(D) + h(D)
= 1 + 2 + 1 + 4 (C & D are AND nodes)
= 8

Since f(A -> B) = 6 is smaller than f(A -> C + D) = 8, we select the path A -> B.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.




## Step 2: Explore Node B

- Next, we explore node **B**, and calculate the values for nodes **E** and **F**:

f(B -> E) = g(E) + h(E)
= 1 + 7
= 8

f(B -> F) = g(F) + h(F)
= 1 + 9
= 10

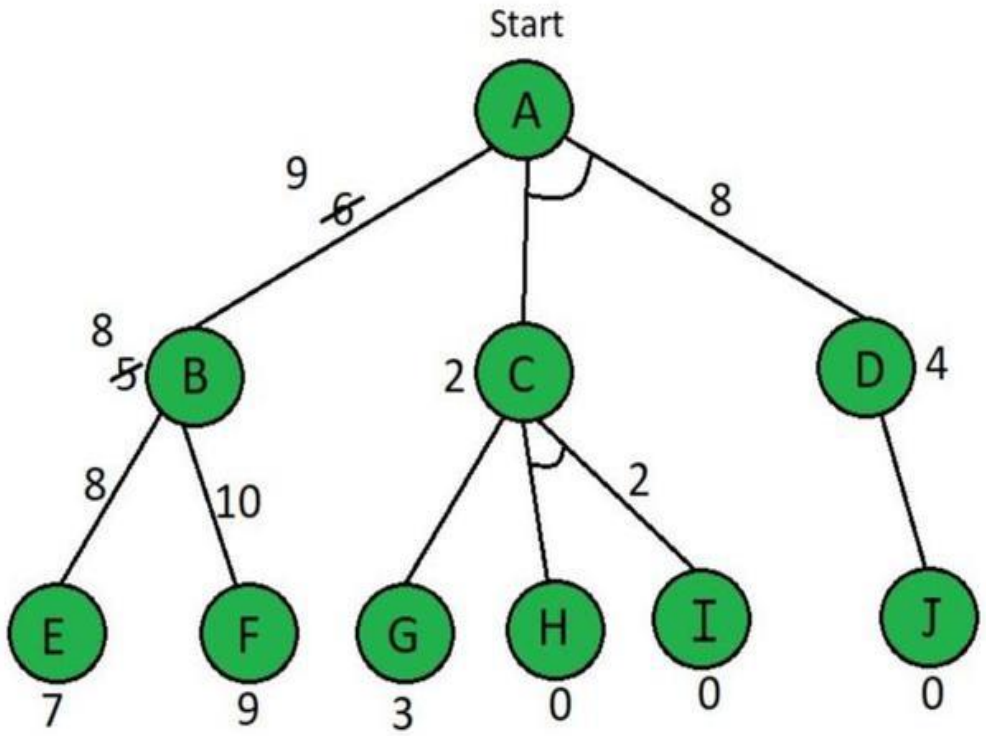
So, by calculation, the path **B → E** is chosen as the minimum path with cost **f(B → E) = 8**. Since B's heuristic differs from its actual cost, the heuristic is updated. Consequently, A's heuristic must also be recalculated due to the change in B's value.

f(A -> B) = g(B) + updated h(B)

= 1 + 8

= 9


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.




Now, we compare **f(A -> B) = 9** with **f(A -> C + D) = 8**. Since **f(A -> C + D)** is smaller, we explore this path and move to node **C**.

For node **C**:

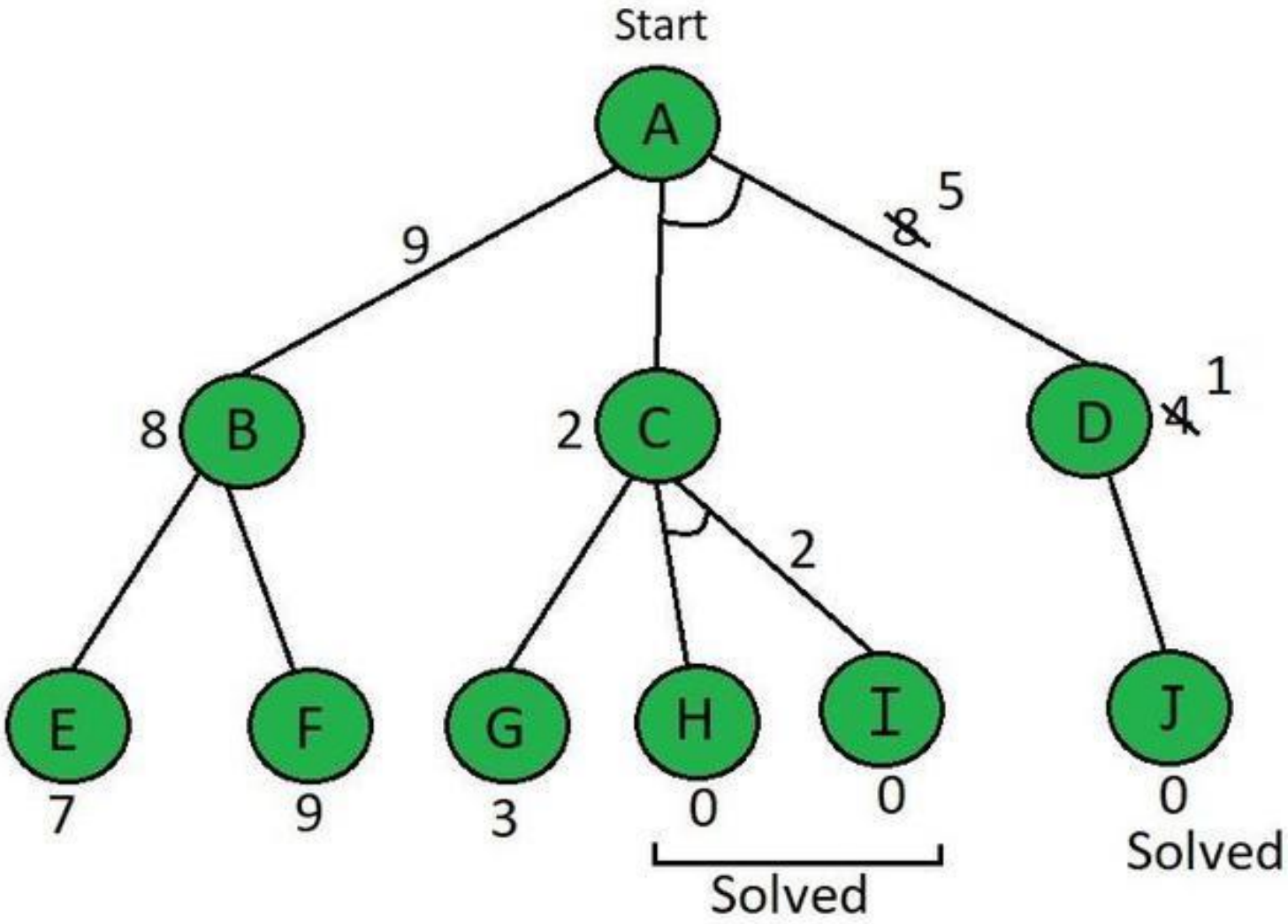
f(C -> G) = g(G) + h(G)
= 1 + 3
= 4

f(C -> H + I) = g(H) + h(H) + g(I) + h(I)
= 1 + 0 + 1 + 0 (H & I are AND nodes)
= 2

## AO* Algorithm (Step-2)

For node **D**:
f(D -> J) = g(J) + h(J)
= 1 + 0 = 1
After updating the heuristic for **D**, we recalculate:
f(A -> C + D) = g(C) + h(C) + g(D) + h(D)
= 1 + 2 + 1 + 1 = 5
Now that **f(A -> C + D)** has the lowest cost, this becomes the solved path, and the **AND-OR graph** is now fully resolved.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.





This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# ***HILL CLIMBING ALGORITHM***

- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem.
- It terminates when it reaches a peak value where no neighbor has a higher value.
- Hill climbing algorithm is a technique which is used for optimizing the mathematical problems.
- One of the widely discussed examples of Hill climbing algorithm is Traveling-salesman Problem in which we need to minimize the distance traveled by the salesman.
- It is also called greedy local search as it only looks to its good immediate neighbor state and not beyond that.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

- A node of hill climbing algorithm has two components which are state and value.
- Hill Climbing is mostly used when a good heuristic is available.
- In this algorithm, we don't need to maintain and handle the search tree or graph as it only keeps a single current state.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

## Types of Hill Climbing Algorithm:

- Simple hill Climbing
- Steepest-Ascent hill-climbing
- Stochastic hill Climbing

## 1. Simple Hill Climbing:

- Simple hill climbing is the simplest way to implement a hill climbing algorithm. **It only evaluates the neighbor node state at a time and selects the first one which optimizes current cost and set it as a current state.** It only checks its one successor state, and if it finds better than the current state, then move else be in the same state. **This algorithm has the following features:**
- Less time consuming
- Less optimal solution and the solution is not guaranteed


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

## 2. Steepest-Ascent hill climbing:

The steepest-Ascent algorithm is a variation of simple hill climbing algorithm. This algorithm examines all the neighboring nodes of the current state and selects one neighbor node which is closest to the goal state. This algorithm consumes more time as it searches for multiple neighbors.

## Algorithm for hill climbing:

Step 1: Define the search space: Determine the range or domain of possible solutions for the problem.

Step 2: Define the objective function: Create a function that measures the quality or fitness of a solution.

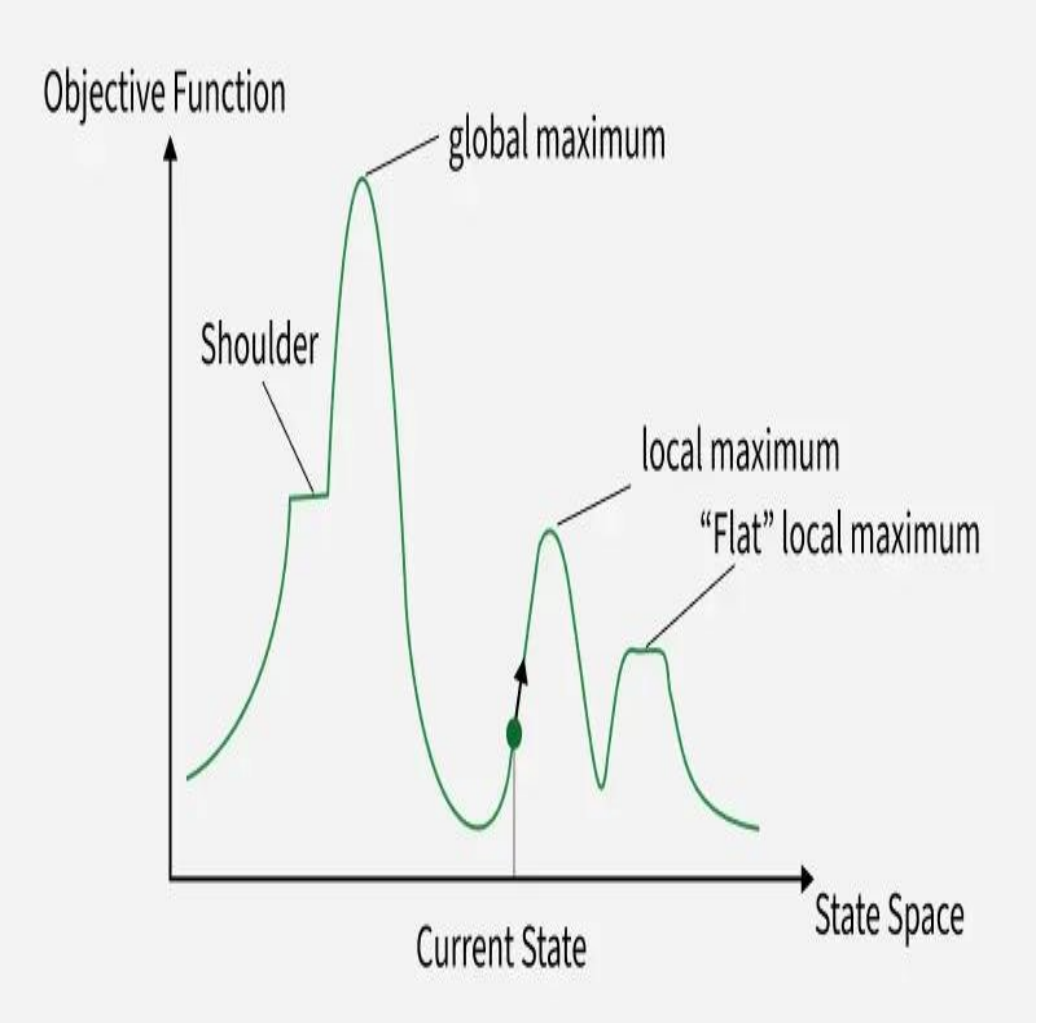
Step 3: Initialization: Generate or select an initial solution within the search space.

Step 4: Iterative process: Repeat the following steps until a termination condition is met:

- Evaluate the current solution using the objective function.
- Generate neighboring solutions by making small modifications to the current solution.
- Select the best neighboring solution based on its objective function value.
- Compare the objective function value of the best neighboring solution with the current solution.
- If the neighboring solution is better, update the current solution. Otherwise, terminate the algorithm.

Step 5: Termination condition: Define a condition to stop the algorithm, such as reaching a maximum number of iterations, achieving a specific threshold for the objective function value, or exceeding a time limit.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.




- Local Maximum: A local maximum is a state better than its neighbors but not the best overall. While its objective function value is higher than nearby states, a global maximum may still exist.
- Global Maximum: The global maximum is the best state in the state-space diagram where the objective function achieves its highest value. This is the optimal solution the algorithm seeks.
- Plateau/Flat Local Maximum: A plateau is a flat region where neighboring states have the same objective function value, making it difficult for the algorithm to decide on the best direction to move.
- Ridge: A ridge is a higher region with a slope which can look like a peak. This may cause the algorithm to stop prematurely, missing better solutions nearby.
- Current State: The current state refers to the algorithm's position in the state-space diagram during its search for the optimal solution.
- Shoulder: A shoulder is a plateau with an uphill edge allowing the algorithm to move toward better


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# ***Simulated Annealing Search***

- Simulated Annealing in AI is a popular optimization technique inspired by the annealing process in metallurgy.
- In metallurgy, annealing involves heating and then slowly cooling a material to remove defects and improve its properties.
- Similarly, in AI, simulated annealing helps in finding the near best solutions to complex problems by exploring different solutions and gradually converging to an optimal or near-optimal solution.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

## How Simulated Annealing Works

- **Initialization**: Begin with an initial solution and an initial temperature. This initial solution can be randomly generated or based on some heuristic.
- **Neighbor Selection**: Generate a neighboring solution by making small random changes to the current solution. Evaluate this new solution using an objective function.
- If the new solution is better (i.e., has a lower objective function value), accept it as the current solution. If it is worse, accept it with a certain probability. This probability decreases as the temperature decreases and is calculated using:

$$\text{Acceptance Probability} = \exp\left(\frac{\text{Current Value} - \text{New Value}}{T}\right)$$

where T is the current temperature.

- **Cooling Schedule**: Gradually decrease the temperature according to a cooling schedule. The cooling rate determines how quickly the temperature drops. A common cooling schedule is:

$$T_{\text{next}} = T_{\text{current}} \times \text{cooling rate}$$

- **Termination:** Continue the process until the temperature falls below a predefined threshold or a maximum number of iterations is reached. The best solution found during the process is then returned as the result.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

The algorithm for simulated annealing is as follows:

1. Let $IS$ be the initial state and $GS$ be the goal state.
2. Check if $IS = GS$. If true, return success.
   Else
   (i) Initialise $t$ for $IS$, where $t$ is temperature or energy level to allow maximum movement
   (ii) Do while a stop condition is not satisfied.
      (a) Randomly pick a neighbour, say $x$.
      (b) Evaluate $x$.
      (c) $\Delta E$ = Energy_val($x$) − Energy_val($IS$)
      (d) If $x = GS$, return success.
      (e) If $\Delta E < 0$, make $x$ to be $IS$. (This means new state/solution is a better one)
      (f) Else generate random number: $r[0, 1]$.
      If $r < e^{-\Delta E/t}$, make $x$ to be $IS$.
      (g) Revise $t$ values.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

# ***Local Beam Search Algorithm***

- Local beam search is a heuristic search algorithm that explores multiple states simultaneously instead of just one.
- Unlike hill climbing (which can get stuck in local optima), beam search tries to improve the chances of reaching the goal by maintaining and exploring **K states** at each step.

**Algorithm Steps**

- Maintain **K states**, not just a single state.
- The search begins with **K randomly generated states**.
- At each iteration, generate **all possible successors** of the K states.
- If the goal state is found, **stop**. Otherwise, select the **K best successors** from the pool.


This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.

## Working

**Initialization:** Start with the root node and generate its successors.

**Node Expansion:** From the current nodes, generate successors and apply the heuristic function to evaluate them.

**Selection:** Select the top W nodes according to the heuristic values. These selected nodes form the next level to explore.

**Iteration:** Repeat the process of expansion and selection for the new level of nodes until the goal is reached or a certain condition is met (like a maximum number of levels).

**Termination:** The search stops when the goal is found or when no more nodes are available to expand.



This file is meant for personal use by dominic097@gmail.com only.
Sharing or publishing the contents in part or full is liable for legal action.